STUDY GUIDE · PREPARATION FOR TESTS AND EXAMS

Computer Science: C programming Weeks 4–7

All rules, questions, and assignments from lectures, labs, and bug hunts of the course — with model answers, code analysis, and solutions for each week: data types, control structures, functions, structures, pointers, strings, and recursion.

Computer Science (C Programming) Course · Fall semester 2026
Tashkent Technological University

QUESTIONS · TESTS · CODE · RULES

OCTOBER 2026

Contents

1. Course map, sources, and how to use this manual . . . 3

1.1. Weeks 4–7 and course files . . . 3

2. Week 4 — Data Types and Operators . . . 4

2.1. Key points ("all rules" of week 4) . . . 4

2.2. Savollar haftasi 4 — lecture and laboratory questions with model answers . . . 9

2.3. Week 4 Tests . . . 10

2.4. Assignments — glossary of week 4 . . . 12

3. Week 5 — Control Structures . . . 13

3.1. Key Points ("all rules" of week 5) . . . 13

3.2. Week 5 Questions — lecture and laboratory questions with model answers . . . 19

3.3. Week 5 Tests . . . 20

3.4. Assignments — glossary for week 5 . . . 22

4. Week 6 — Functions and Structures . . . 22

4.1. Key Points ("all rules" of week 6) . . . 23

4.2. Week 6 Questions — lecture and lab questions with model answers . . . 29

4.3. Week 6 Tests . . . 30

4.4. Assignments — glossary of week 6 . . . 32

5. Week 7 — Pointers, Strings, and Recursion . . . 33

5.1. Key points ("all rules" of week 7) . . . 33

5.2. Week 7 questions — lecture and lab questions with model answers . . . 39

5.3. Week 7 Tests . . . 41

5.4. Assignments — Week 7 Glossary . . . 43

6. Bug Hunts — all errors and their fixes 43

6.1. Bug Hunt C 44

6.2. Bug Hunt Loops 48

6.3. Bug Hunt Pointers & Strings 53

7. Cheat sheet: key rules and tables 57

7.1. Operator precedence 57

7.2. printf/scanf formats 58

7.3. Key rules of C 59

7.4. Ten answers in one line 60

7.5. What's next 60

1. Course map, sources, and how to use this manual

This manual is compiled from all files of the **Computer Science — programming in C** course (track **Foundations of Computer Science, FoCS**; fall semester 2026, Tashkent University of Technology). The course consists of four layers: (1) weekly lectures — FoCS/TashTech Lectures 4–7 (October 6, 13, 20, and 27, 2026) — review of the previous topic and a new topic; (2) laboratory exercises based on chapters 5–8 of the textbook by Ceri, Mandrioli & Sbattella «The Art and Craft of Computing» in C11-adaptation with CS50 input helpers (get_{double}, get_{char}, get_{string}); (3) **bug hunt** worksheets — "bug hunting" on conditional constructs and functions, loops, pointers, and strings; (4) **coffee machine** project files (week 6 model: Stock, Recipe, Cup). The manual covers the material of weeks 4–7: data types and operators, control structures, functions and structures, pointers, strings, and recursion.

Every savol (savollar — questions) from lectures and labs, every test (testlar), every atama (atamalar — terms), and every code fragment from slides, labs, and bug hunts is fully transcribed and provided with a model answer or solution. All code examples are taken verbatim from the course files; where the PDF extraction listing was "broken" with spaces, it has been restored according to the slide's meaning. The original instructor formulations are preserved: «Pop quiz», «Prediction», «Exit check», «Expected».

How to use the manual: (1) Start with the "Key Points" section of each week — this is all the "rules" (qoidalar) of the week in a condensed form with tables; (2) Work through the open questions (savollar) aloud, covering the answers, and check against the model answers; (3) Complete all tests (testlar) — the same formulations are used on exams; (4) Go through all the code line by line: on the exam, they ask "what will this program print?"; (5) Learn the glossary atamalar — terms appear in definition questions; (6) Before the exam, review all bug hunts from chapter 6 and the cheat sheet from chapter 7 (operator precedence, printf/scanf formats, ten answers in one line).

1.1. Weeks 4–7 and course files

Each week corresponds to a set of files in the course folder; all of them have been reworked in this manual. Lectures are on Tuesdays, labs are by textbook chapter (chapter 5 — week 4, chapter 6 — week 5, chapter 7 — week 6, chapter 8 — week 7).

Week	Topic	Lectures/Lab	Additional files
4	Data Types and Operators	$lecture_4$.txt — FoCS Lecture 4 (October 6, 2026): review, variables and data types, operators epilogue · $Week_4$.txt — lab "Programming in C — Data Types and Operators" (textbook chapter 5)	—
5	Control Structures	$Lecture_5$.txt — Lecture 5 (October 13, 2026): blocks, conditions, loops, functions, scopes · Week-5-Control-Structures.txt — lab "Chapter 6 · Control structures"	bug_{huntc}.txt (tasks 1–4: branching) · $bug_{huntloops}$.txt
6	Functions and Structures	$Lecture_6$.txt — TashTech Lecture 6 (October 20, 2026): control flow, I/O, strings, files, command line · Week-6-Functions-and-Structures.txt — lab "WEEK 6 · CHAPTER 7": from Scratch blocks to coffee machine	bug_{huntc}.txt (tasks 5–6: functions) · project "coffee machine" (Stock, Recipe, Cup)

Week	Topic	Lectures/Lab	Additional files
7	Pointers, Strings, Recursion	Lecture$_{71}$.txt — TashTech Lecture 7 (October 27, 2026): pointers and addresses, arrays and address arithmetic, strings, search and sorts · Recursion-Lecture.txt — "WEEK 7 · CHAPTER 8: Recursion"	bug$_{huntpointersstrings}$.txt · bug$_{huntpointersstringsanswerkey}$.txt (instructor key)

Weeks 4–7: topics and course files

2. Week 4 — Data Types and Operators

Week files: Week$_4$.txt (lab "Programming in C — Data Types and Operators", chapter 5) · lecture$_4$.txt (FoCS Lecture 4 — October 6, 2026)

Week 4 — the foundation of the entire course: C variables and data types, their sizes and precision, integer and floating-point division, arithmetic, logical, bitwise operators, increment/decrement, compact assignment, ternary operator, type conversions and operator precedence. Week's laboratory: sphere precision (sphere$_{precision}$), fixing leap year rule (leap$_{year}$), bridge to memory (bytes and character arrays) and alphabetical join (alphabetical$_{join}$); in a separate LMS session — Circle precision and Character processor tasks, and five-letter join — challenge. Below are all the rules, questions, tests and terms of the week.

2.1. Key points ("all rules" of week 4)

C data types: declaration, variable names, constants

C is a fast, small, universal, platform-independent language; used for systems programming (compilers and interpreters, operating systems, DBMS, microcontrollers). C is static (compiled), typed, structured and imperative. "C is quirky, flawed, and an enormous success" — Ritchie. **Data type** of an object in memory determines the set of values it can take and the operations that can be applied to it; C is a weakly typed language: it allows implicit conversions and forced (potentially dangerous) type casting.

The family of C types is small: numeric (int, float, double), character (char), user-defined (struct, union). Basic laboratory table:

Type	C Declaration	Typical use
int	int score = 72;	integers
char	char grade = 'B';	one basic character
float	float temperature = 23.5f;	approximate real values
double	double radius = 3.0;	approximate real values
bool	bool passed = true;	true or false
string	string name = "Ada";	text (via CS50)

Variables and Types [Source: Week$_4$, slide 2]

Slide clarifications: in C11, bool is a macro from <stdbool.h>; string is a CS50-typedef for char * (stores the address, not the text itself); one char is one byte, a "basic character" like B, not any Unicode character; a variable of type void cannot be declared. **int main(void)**: int is the return type, void in parentheses means "no parameters"; **void example(void)**: the first void means "no return value", the second means "no parameters".

General declaration format: **type variable-name [= value]**. Lecture examples:

```
char x;                /* uninitialized */
char x = 'A';          /* initialized to 'A' */
char x = 'A', y = 'B'; /* multiple variables initialized */
char x = y = 'Z';      /* multiple initializations */
```

[Source: lecture$_4$, slide 12]

Variable naming rules: names can contain letters, numbers, and _; must start with a letter; cannot use keywords (for, while, etc.); names are case-sensitive — int x; and int X; declare two different variables. Lecture pop quiz: int money$owed; — invalid (character $); int total$_{count}$; — valid; int score2; — valid; int 2ndscore; — invalid (starts with a digit); int long; — invalid (keyword).

Constants — literal/fixed values. Integer: int i = 3; long l = 3; unsigned long ul = 3UL; hexadecimal int i = 0xA;; octal int i = 012;. Floating-point: float pi = 3.14159; float pi = 3.141F; double pi = 3.14159265358979323 84L;. Character: 'A', '\x41' (hex), '\0101' (octal). String: "hello world"; "hello" "world" — same as "hello world". Enumerations: enum BOOL {NO, YES} → NO = 0, YES = 1; enum COLOR {R = 1, G, B, Y = 10} → G = 2, B = 3 — an enumerator without an explicit value continues numbering from the previous one.

[Source: lecture$_4$, slides 10–11; Week$_4$, slide 2]

Sizes, ranges, and precision: sizeof, float and double

Individual sizes depend on the machine/compiler, but guaranteed: sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long) and sizeof(char) < sizeof(short) <= sizeof(float) <= sizeof(double). The unsigned version has approximately twice the range of the signed version; signed and unsigned char differ only in arithmetic expressions. For multi-byte numbers, byte order is important: **big endian** — most significant bits (MSBs) at the lowest address (powerpc processors; network order); **little endian** — least significant bits (LSBs) at the lowest address (all x86-compatible processors). This is the "NUXI problem".

Type	signed	unsigned
short	short int x; short y;	unsigned short x; unsigned short int y;
by default	int x;	unsigned int x;
long	long x;	unsigned long x;
float	float x;	N/A
double	double x;	N/A
char	char x; signed char x;	unsigned char x;

Numeric types: signed and unsigned [Source: lecture$_4$, slide 7]

Laboratory tables (measured on macOS arm64, 8 bits per byte; values may differ on other platforms):

Type	Bits	Range
char	8	–128 … 127
signed char	8	–128 … 127
unsigned char	8	0 … 255
short	16	–32 768 … 32 767
unsigned short	16	0 … 65 535
int	32	–2 147 483 648 … 2 147 483 647
unsigned int	32	0 … 4 294 967 295
long	64	–9 223 372 036 854 775 808 … 9 223 372 036 854 775 807

Type	Bits	Range
unsigned long	64	0 … 18 446 744 073 709 551 615
long long	64	–9 223 372 036 854 775 808 … 9 223 372 036 854 775 807
unsigned long long	64	0 … 18 446 744 073 709 551 615

Integer types: sizes and ranges; signed int = int, short int = short, long int = long [Source: Week$_4$, slides 27–28]

Type	Bits	Decimal precision
float	32	6 significant digits
double	64	15 significant digits
long double	64	15 significant digits (like double on this platform)

Real types: precision from FLT_{DIG}, DBL_{DIG}, $LDBL_{DIG}$ (<float.h>) [Source: Week$_4$, slide 29]

Laboratory conclusions: **float** has 6, **double** has 15 significant decimal digits (significant digits, not decimal places); for sphere radius and volume, we use **double**. Sphere problem:

$$V = (4 / 3) \times \pi \times r^3$$

```
const double pi = 3.1415;
double radius = get_double("Radius: ");
// Today's input range: 0 < radius <= 1000
```

[Source: Week$_4$, slide 4]

Division and remainder: the main trap of the week

Lecture rule: **types of operands, not the receiver type, determine how division works**. float x = 3/2; gives x = 1 (integer division); float x = 3.0/2; gives x = 1.5 (real division); int x = 3.0/2; again gives 1 (conversion to int). Remainder: int x = 3%2; → 1; int y = 7; int x = y%4; → 3; int x = y%10; → 7.

Division is performed **before** assignment — laboratory prediction table:

Expression	Stored in double	Printed with %f
4 / 3	1.0	1.000000
4.0 / 3.0	≈ 1.333333…	1.333333
(double)(4 / 3)	1.0	1.000000

Types of operands determine how / works [Source: Week$_4$, slide 6]

```
double fraction = 4 / 3;
printf("%f\n", fraction);
```

Laboratory Prediction 1: this fragment prints **1.000000** (option A) — integer division gives 1, and only then assignment converts 1 to double. Does the receiver type change the execution of 4/3? No.

Formatting does not improve the stored value: at radius = 3, the same correct volume is printed as **113.1** (%.1f) and **113.0940** (%.4f) — printf only changes the representation, not the stored number. Valid radius: strictly greater than 0 and not greater than 1000; zero, negative values, and everything over 1000 are rejected by the program (positive range check also rejects non-numeric input).

[Source: Week$_4$, slides 3–8; lecture$_4$, slide 15]

Comparison and logical operators: short circuit

Comparison operators compare two operands and yield a "boolean" result: in C, **any non-zero value (by convention 1) is true, 0 is false**. Lecture examples: 3 > 2 → 1; 2.99 > 3 → 0; 3 >= 3 → 1; 3 < 3 → 0; 'A' < 'B'

→ 1; 3 == 3 → 1; 'A' == 'a' → 0; 3 != 3 → 0; 2.99 != 3 → 1. Traps: **== (equality) is not = (assignment)**; comparison == of real variables is unreliable due to finite precision.

Logical operators && (AND), || (OR), ! (NOT):

```
((9/3) == 3) && (2*3 == 6); /* evaluates to 1 */
('A' == 'a') && (3 == 3)    /* evaluates to 0 */
2 == 3 || 'A' == 'A';       /* evaluates to 1 */
2.99 >= 3 || 0              /* evaluates to 0 */
!(3 == 3);                  /* evaluates to 0 */
!(2.99 >= 3)                /* evaluates to 1 */
```

Short circuit: expression evaluation stops if the value of the condition is already determined; watch for side effects. Lecture examples: (3 == 3) || ((c = getchar()) == 'y') — the second part is NOT evaluated; (0) && ((x = x + 1) > 0) — the second part is NOT evaluated, x will not change. Laboratory table (array boundary before character access):

i	i < 3	Read characters?
0, 1, 2	true	yes
3	false	no

i < LENGTH && first[i] == second[i]: && skips the right part when the left is false [Source: Week$_4$, slide 22]

[Source: lecture$_4$, slides 16–18; Week$_4$, slide 22]

Increment/decrement, compact assignment and ternary operator

Postfix: x++ is shorthand for x = x+1; y = x++ is equivalent to y = x; x = x+1; — x is evaluated **before** increment (similarly for x−−). **Prefix**: ++x; y = ++x is equivalent to x = x+1; y = x; — x is evaluated **after** increment. Compact assignment operators for the pattern var = var (op) expr:

```
x += 1   /* is the same as x = x+1 */
x -= 1   /* is the same as x = x-1 */
x *= 10  /* is the same as x = x*10 */
x /= 2   /* is the same as x = x/2 */
x %= 2   /* is the same as x = x%2 */
```

Ternary operator ?: — syntactic sugar for the pattern if (cond) x = <expra>; else x = <exprb>;:

```
sign = x > 0 ? 1 : -1;
/* if (x > 0)
       sign = 1;
   else
       sign = -1; */

isodd = x % 2 == 1 ? 1 : 0;
/* if (x % 2 == 1)
       isodd = 1;
   else
       isodd = 0; */
```

[Source: lecture$_4$, slides 19–23]

Bitwise operators

Operator	Meaning	Examples
&	AND	0x77 & 0x3 → 0x3; 0x77 & 0x0 → 0
|	OR	0x700 | 0x33 → 0x733; 0x070 | 0 → 0x070
^	XOR	0x770 ^ 0x773 → 0x3; 0x33 ^ 0x33 → 0

Operator	Meaning	Examples
<<	shift left	0x01 << 4 → 0x10; 1 << 2 → 4
>>	right shift	0x010 >> 4 → 0x01; 4 >> 1 → 2

Bitwise operators [Source: lecture$_4$, slide 21]

Slide rules: AND is true only if both operands are true; OR — if at least one; XOR — if exactly one.

Type conversions and operator precedence

When promoted to higher precision, data is preserved — the compiler does this automatically for expressions with mixed types; rule (with exceptions): each member of a binary expression is promoted to the precision of the highest precision operand. Automatic conversion char → int allows comparing and "calculating" characters:

```
int i;
float f;
f = i + 3.14159; /* i is promoted to float,
                    f = (float)i + 3.14159 */

isupper = (c >= 'A' && c <= 'Z') ? 1 : 0;
/* c and literal constants are converted to int */
if (!isupper)
    c = c - 'a' + 'A';
/* subtraction is possible because of integer conversion */
```

Precedence and order of evaluation: ++, −−, (cast), sizeof — highest precedence; *, /, % higher than +, −; ==, != higher than &&, ||; assignment operators — very low. Use parentheses ( ) generously to avoid ambiguities and side effects of precedence:

```
y = x*3+2            /* same as y = (x*3)+2 */
x != 0 && y == 0     /* same as (x != 0) && (y == 0) */
d = c >= '0' && c <= '9'  /* same as d = (c >= '0') && (c <= '9') */
```

[Source: lecture$_4$, slides 24–25]

Laboratory: leap year and character arrays

Leap year. Common year — 365 days (February has 28), leap year — 366 (February 29): the extra day keeps the calendar in agreement with the seasons. The rule is read top to bottom: divisible by 400? → leap year; otherwise divisible by 100? → common; otherwise divisible by 4? → leap year; otherwise common. 2000 — leap year (divisible by 400); 1900 — common (divisible by 100 but not 400); 2024 — leap year; 2025 — common; 2100 — common (exit check: divisible by 100 but not 400). Compact laboratory expression:

```
year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)
```

Repeatable input ("Go back to line 1" — CS50-style repetition pseudocode): read year before the loop and again at the end of the while loop; year 0 or negative terminates without classification; greater than 9999 — error message and stop. If you forget to read again — positive year will never change, the loop will repeat forever. The first rule year % 4 == 0 classifies 2024 and 2025 correctly — these two tests don't reveal the century bug; check 1900 and 2000.

Bridge to memory. RAM module → one chip → conceptual grid: each square is one byte; sizeof(char) — 1 byte in C, number of bits gives $CHAR_{BIT}$ (on the educational machine 8); measured size of int — 4 bytes. Arrays of three characters:

```
char first[3]  = {'d', 'o', 'g'};
char second[3] = {'c', 'a', 't'};
```

Index	0	1	2
first	d	o	g
second	c	a	t

Indices 0–2 · no terminator '\0' · these are not C-strings [Source: Week$_4$, slide 18]

Alphabetical gluing (dog + cat → catdog): compare the two arrays entirely by the first differing character (car/cat differ at index 2: r and t), then copy the "smaller" array first. Scan of equal prefix: i < LENGTH && first[i] == second[i] — the boundary must stand before accessing the character; equal arrays reach the boundary: first$_{\text{earlier}}$ = i == LENGTH || first[i] < second[i]. Copying: i fills joined[0..2], LENGTH + i — joined[3..5]:

Index	0	1	2	3	4	5
joined	c	a	t	d	o	g

Copy offset: i fills 0–2, LENGTH + i fills 3–5 [Source: Week$_4$, slide 23]

Printing: there's no final '\0' in joined, so %s is unsuitable — we print exactly six characters via **%c**.

[Source: Week$_4$, slides 9–26]

2.2. Savollar haftasi 4 — lecture and laboratory questions with model answers

S1. Laboratory warm-up: what types store integers, decimal values, and a single character?

Answer. int — for integers; float or double — for decimal (real) values; char — for a single character. Execution starts with main; stdio.h declares printf, cs50.h — get$_{\text{double}}$.

S2. What does void mean in int main(void) and in void example(void)?

Answer. In int main(void) int is the return type, and void in parentheses means "no parameters". In void example(void) the first void is "no return value", the second is "no parameters". A regular variable of type void cannot be declared.

S3. Why was double chosen for the radius and volume of a sphere, not float?

Answer. The radius can have a fractional part, and on our platform double is more precise than float: 15 significant decimal digits versus 6 (DBL$_{\text{DIG}}$ versus FLT$_{\text{DIG}}$). At the same time, double doesn't make the approximation pi = 3.1415 exact — real values remain approximate.

S4. What will double fraction = 4 / 3; printf("%f\n", fraction); print?

Answer. 1.000000. Division occurs before assignment: both operands are integers, 4/3 gives 1, and only then is 1 converted to double. The receiver type doesn't change how division is performed — the operand types determine it. Similarly, (double)(4/3) gives 1.0, while 4.0/3.0 gives 1.333333.

S5. The sphere program prints the same volume for radius = 3 with both %.1f and %.4f. Do extra digits improve the stored value?

Answer. No. The outputs are 113.1 and 113.0940: these are the same stored number; formatting only changes the representation (printf precision controls the display, not the stored value).

S6. Pop quiz II from lecture: int x = 017; int y = 12; — is x greater than y?

Answer. Yes. 017 is an octal literal (like int i = 012; from the constants table — octal number): $017_8 = 1 \cdot 8 + 7 = 15$, and $15 > 12$.

S7. What is XL in enum sz {S = 0, L = 3, XL}; and in enum sz {S = 0, L = −3, XL};?

Answer. An enumerator without an explicit value continues numbering from the previous one — as in enum COLOR {R = 1, G, B, Y = 10}, where G = 2, B = 3. Therefore, in the first case XL = 4, in the second XL = −2.

S8. Why does 'A' < 'B' give 1, while 'A' == 'a' gives 0?

Answer. char is automatically converted to int (the character code). The codes for 'A' and 'B' are in ascending order, so comparing the codes gives 1; the codes for 'A' and 'a' are different, so equality is false. This same conversion allows for arithmetic like c = c - 'a' + 'A'; (converting lowercase to uppercase).

S9. What is short-circuit evaluation and what happens to x in (0) && ((x = x + 1) > 0)?

Answer. Expression evaluation stops if the result is already determined. The left operand of && is 0, so the entire expression is false, and the right part — with the side effect x = x + 1 — is not evaluated at all: x won't change. Similarly, in (3 == 3) || ((c = getchar()) == 'y'), getchar won't be called. In the loop i < LENGTH && first[i] == second[i], when i = 3, access to first[3] doesn't occur.

S10. What's the difference between y = x++ and y = ++x?

Answer. Postfix: y = x++; is equivalent to y = x; x = x + 1; — x is evaluated before increment. Prefix: y = ++x; is equivalent to x = x + 1; y = x; — x is evaluated after increment. For decrement — similarly: y = x−− and y = −−x.

S11. How does the ternary operator work in sign = x > 0 ? 1 : −1;?

Answer. This is syntactic sugar for if (x > 0) sign = 1; else sign = −1;: the condition is before ?, the value when true is between ? and :, when false is after :. The code becomes shorter and clearer; the second example from the slide is isodd = x % 2 == 1 ? 1 : 0;.

S12. What input should the sphere volume program reject?

Answer. Zero, negative radii, and values greater than 1000: the valid range is 0 < radius <= 1000. Checking the positive range also rejects non-finite input.

S13. Why is 1900 a common year and 2000 a leap year? Why is 2100 common?

Answer. 2000 is divisible by 400 — leap year. 1900 is divisible by 100 but not 400 — common year; 2100 is the same (divisible by 100 but not 400), and the rule checks divisibility by 400 before divisibility by 100. Compact condition: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0); 2024 is leap by this, 2025 is common.

S14. Why can't the char array first[3] = {'d', 'o', 'g'} be printed with %s, and what does the order in && have to do with it?

Answer. The array has exactly three characters and no terminator '\0' — it's not a C-string: %s would continue reading memory beyond the array until a random zero byte. Print exactly three characters with %c. And in the comparison loop, the boundary check must come first: i < LENGTH && first[i] == second[i] — when i = 3, the left part is false, && skips the right part, and no array bound violation occurs.

2.3. Week 4 Tests

T1. What C type stores one basic character?

a) int
b) char
c) double
d) bool

Answer: b) char

Explanation: char grade = 'B'; — one basic character; int — integers, double — approximate real numbers, bool — true/false.

T2. How many significant decimal digits does double have on the course platform?

a) 6
b) 8
c) 15
d) 32

Answer: c) 15

Explanation: $DBL_{DIG} = 15$ (for float — 6, FLT_{DIG}). These are significant digits, not the number of decimal places; long double on this platform also has 15.

T3. What will double fraction = 4 / 3; printf("%f\n", fraction); print?

a) 1.000000
b) 1.333333
c) compilation error
d) 0.000000

Answer: a) 1.000000

Explanation: 4/3 is integer division, giving 1; conversion to double happens on assignment. The correct answer is prediction 1 from the lab — option A.

T4. Which notation represents the integer 15?

a) int x = 0xA;
b) int x = 017;
c) int x = 15.0;
d) int x = '15';

Answer: b) int x = 017;

Explanation: 017 is an octal literal: 1·8 + 7 = 15. 0xA is hexadecimal 10; '15' is not a character (only one character is allowed in single quotes).

T5. Which variable declaration is correct?

a) int money$owed;
b) int 2ndscore;
c) int $total_{count}$;
d) int long;

Answer: c) int $total_{count}$;

Explanation: pop quiz from lecture: $ is not allowed; name cannot start with a digit; long is a keyword. Letters, digits, and _ are allowed, but the first must be a letter.

T6. What is y % 10 when int y = 7;?

a) 0
b) 3
c) 7
d) 1

Answer: c) 7

Explanation: int y = 7; int x = y % 10; / produces 7 / — 7 is less than 10, the remainder equals the number itself; while y % 4 gives 3.

T7. What is 0x01 << 4?

a) 0x10
b) 0x01
c) 4
d) 0x14

Answer: a) 0x10

Explanation: left shift by 4 bits: 0x01 << 4 evaluates to 0x10; similarly 1 << 2 → 4, 0x010 >> 4 → 0x01, 4 >> 1 → 2.

T8. Which group of operators has the highest precedence?

a) = and +=
b) && and ||
c) * and /
d) ++, −−, (cast), sizeof

Answer: d) ++, −−, (cast), sizeof

*Explanation: ++, −−, (cast), sizeof have the highest precedence; then *, /, %; then +, −; ==, != are higher than &&, ||; assignment has very low precedence.*

T9. The expression y = x*3 + 2 is equivalent to…

a) y = x*(3+2)
b) y = (x*3)+2
c) y = x+(3*2)
d) y = 3*(x+2)

Answer: b) y = (x*3)+2

Explanation: , /, % have higher precedence than + and −, so multiplication is performed first — as in the slide example "same as y = (x3)+2".

T10. What happens to x in the expression (0) && ((x = x + 1) > 0)?

a) x will increase by 1
b) x will not change
c) compilation error
d) x will become 0

Answer: b) x will not change

Explanation: short circuit: the left operand of && is already false, the right part with side effects is not evaluated — "be careful of any side effects".

2.4. Assignments — glossary of week 4

Term (atama)	Meaning
Data type (datatype)	defines the set of values for an object in memory and the permissible operations on it; C is a weakly typed language with implicit conversions and casting
Variable	a named binding/reference to a value stored in memory, or a computable expression (int x = 0, y = 0; y = x + 2;)
Expression	a combination of values, variables, operators, and functions (y = x + 2)
Operator	specifies manipulation of an object; unary (–, ++), binary (+, –, *, /), ternary (?:)

Term (atama)	Meaning
Integer division	division of two integers: the fractional part is discarded (4/3 → 1, 3/2 → 1); the type of the receiver doesn't matter — the types of the operands are important
Remainder (modulus, %)	remainder of integer division: 3 % 2 → 1, 7 % 4 → 3, 7 % 10 → 7
Significant digits	precision of floating-point type: float — 6, double — 15 significant decimal digits (FLT_{DIG}, DBL_{DIG} from <float.h>)
Byte order (endianness)	big endian — most significant bytes at lowest address (powerpc, network order); little endian — least significant at lowest (x86); "NUXI problem"
Constant	literal/fixed value: 3, 3UL, 0xA (hex), 012 (octal), 'A', '\x41', "hello world"
Enumeration (enum)	named integer constants; without explicit values, numbering continues: enum COLOR {R=1,G,B,Y=10} → G=2, B=3
Short circuit	termination of evaluation of && and || when the result is already determined; side effects of the right part may not be executed
Operator precedence	order: ++/−−/(cast)/sizeof → *,/,% → +,− → ==,!= → &&,|| → assignment; parentheses ( ) are generous
Increment/decrement	x++ (postfix: value before increment) and ++x (prefix: value after increment)
Leap year	a year divisible by 400, or divisible by 4 and not by 100: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)

3. Week 5 — Control Structures

Week files: Week-5-Control-Structures.txt (lab "Chapter 6 · Control structures") · $Lecture_5$.txt (Lecture 5 — October 13, 2026)

Week 5 teaches the program to "make decisions and repeat tasks": blocks and compound statements, conditions (if / else if / else, switch), loops (while, for, do-while), break and continue, and in lecture — also functions, modular programming and scope (static, register, extern). The lab covers chapter 6 of the textbook: Melody example (melody from a name), string length calculation in CS50 style, optional do-while and continue fragments. Below are all the rules, questions, tests, and terms of the week.

3.1. Key Points ("all rules" of week 5)

Blocks and compound statements

A simple statement ends with a semicolon: z = foo(x+y);. Multiple statements are combined into a compound statement (block) with curly braces. A block can stand where a simple statement is expected; it is compiled as a single unit; variables can be declared inside; a block can be empty {}; after the closing brace, no semicolon is placed. Blocks can be nested inside each other:

```
{
    int temp = x + y;
    z = foo(temp);
    {
        float temp2 = x * y;
        z += bar(temp2);
    }
}
```

[Source: $Lecture_5$, slides 4–6]

Condition in C and the if / else if / else family

Unlike C++ or Java, C89/C90 has no boolean type; in C99 bool is available via <stdbool.h>. **A condition is an expression (or a series of expressions)**, for example n < 3 or x < y || z < y. A non-zero expression ⇒ the condition is true; the expression must be numeric (or a pointer):

```
const char str[] = "some text";
if (str) /* string is not null */
    return 0;
```

if: evaluate the condition; if true — execute the inner statement; otherwise — do nothing. **else** (optional): execute the statement if the condition is false. **else if**: additional alternative paths; conditions are checked in order until the first true one — if several are true, only the first will execute; equivalent to nested ifs. Any inner statement can be a block:

```
if (x % 2) y += x / 2;

if (x % 2 == 0)
    y += x/2;
else
    y += (x+1)/2;

if (x % 2 == 0) y += x / 2;
else if (x % 4 == 1) y += 2 * ((x + 3) / 4);
else
    y += (x + 1) / 2;
```

[Source: Lecture$_5$, slides 7–11]

Nested if — the classic pitfall: else without braces associates with the **nearest inner** if. To attach else to the outer if — use curly braces:

```
if (x % 4 == 0)
    if (x % 2 == 0) y = 2;
    else
        y = 1;
/* else belongs to the inner if! */

if (x % 4 == 0) {
    if (x % 2 == 0) y = 2;
} else
    y = 1;
/* now else is connected to the outer if */
```

[Source: Lecture$_5$, slides 12–13]

switch: case, break, default and fall-through

switch is an alternative conditional operator: it takes an integer (or character) variable at input; cases (case) for its value are considered. The switch content is a block; **case labels are different entry points into the block**; the compiler compares the variable with each label in order; on a match, it executes the inner code **until the nearest break**; without break, execution "falls through" to the next case without rechecking; default is the "otherwise" branch. Lab C principles: switch takes an integer expression (including char); case labels are different integer constant expressions; for ranges and compound conditions, use if. Full lab example (grade):

```
if (grade == 'A') {
    printf("Perfect!\n");
} else if (grade == 'B') {
    printf("You did good!\n");
} else if (grade == 'C') {
    printf("You did okay!\n");
} else if (grade == 'D') {
```

```
    printf("At least it's not an F!\n");
} else if (grade == 'F') {
    printf("YOU FAILED!\n");
} else {
    printf("That's not a valid grade!\n");
}
```

```
switch (grade) {
case 'A':
    printf("Perfect!\n");
    break;
case 'B':
    printf("You did good!\n");
    break;
case 'C':
    printf("You did okay!\n");
    break;
case 'D':
    printf("At least it's not an F!\n");
    break;
case 'F':
    printf("YOU FAILED!\n");
    break;
default:
    printf("That's not a valid grade!\n");
}
```

Each equality check from the if/else-if ladder becomes a case label, else becomes default; break exits the switch (execution continues after the closing brace — break does not terminate the program). Both programs give the same messages for all grades A/B/C/D/F and W. Multiple case labels are placed one after another:

```
switch (ch) {
case 'Y':  /* ch == 'Y' */
    /* do something */
    break;
case 'N':  /* ch == 'N' */
    /* do something else */
    break;
default:   /* otherwise */
    /* do a third thing */
    break;
}
```

```
switch (ch) {
case 'Y':
case 'y':
    /* do something if ch == 'Y' or ch == 'y' */
    break;
}
```

[Source: Week-5-Control-Structures, slides 5–6; Lecture5, slides 14–15]

Example 6.1: melody from a name

Book Example 6.1 reads one character at a time, calculates the ASCII code modulo 7, and prints the note name (sound is not played). For 'A': code 65; 65 % 7 = 2; expected note is C. In the lab version 01$_{\text{melodybuggy}}$.c after case 2 **break is missing** — this is an intentional bug for analysis: input A# prints C, then D (falling through to case 3 without rechecking). Analysis in debug50: breakpoint on printf case 2, Step Over to case 3; fix — add break after printf in case 2; test with B# and immediate #. The fixed version reads a character, checks for successful input and the sentinel character # (scanf(" %c", &C) skips whitespace characters), selects a note and

reads again; EOF and # terminate the loop; break inside case leaves the switch, so new reading occurs. Input AB# gives note C, then note D.

Character	ASCII code	65 % 7	Expected note
A	65	2	C

Melody: 'A' → code 65 → remainder 2 → note C [Source: Week-5-Control-Structures, slide 7]

Loops: while, for, do-while

while — the simplest loop: evaluate the body while the condition is true. The condition is evaluated **before** the body, so the body may not execute at all:

```
while (/* condition */)
    /* loop body */
```

for — a "counting" loop: in parentheses, three expressions separated by semicolons — initialization (i = 1), condition (i <= n), increment (i++). Expressions can be empty (empty condition is considered true). Execution order: initialization once; condition before each body; body; update; condition again. Equivalent to while and a variant with the comma operator (comma is a low-priority operator, evaluated left to right; this is not the same as comma between function arguments):

```
int factorial(int n) {
    int i, j = 1;
    for (i = 1; i <= n; i++) j *= i;
    return j;
}

int factorial(int n) {
    int j = 1;
    int i = 1; /* initialization */
    while (i <= n /* condition */) {
        j *= i;
        i++; /* increment */
    }
    return j;
}

int factorial(int n) {
    int i, j;
    for (i = 1, j = 1; i <= n; j *= i, i++)
        ;
    return j;
}
```

Laboratory comparison (§6.2.1): both fragments print **1 2 3**; the while version declares the counter int i = 1; before the loop, the for version in the header, and this counter is visible only inside the loop (don't refer to it after):

```
int i = 1;
while (i <= 3)
{
    printf("%d ", i);
    i++;
}

for (int i = 1; i <= 3; i++)
{
    printf("%d ", i);
}
```

do-while differs from while in that the condition is evaluated **after** each iteration: the body is executed at least once; note the semicolon at the end:

```
char c;
do {
    /* loop body */
    puts("Keep going? (y/n) ");
    c = getchar();
    /* other processing */
} while (c == 'y' /* other conditions */);
```

[Source: Lecture 5, slides 18–22; Week-5-Control-Structures, slides 10–11]

break and continue

Sometimes you need to terminate a loop prematurely. **break;** exits the innermost loop or switch statement:

```
char c;
do {
    /* loop body */
    puts("Keep going? (y/n) ");
    c = getchar();
    if (c != 'y')
        break;
    /* other processing */
} while (/* other conditions */);
```

continue; skips the rest of the innermost loop body and goes to its condition. The lecture example is GCD with a min macro: if i doesn't divide a, we go to the next i; if i divides both a and b — we remember:

```
#define min(a,b) ((a)<(b)?(a):(b))

int gcd(int a, int b) {
    int i, ret = 1, minval = min(a, b);
    for (i = 2; i <= minval; i++) {
        if (a % i) /* i not divisor of a */
            continue;
        if (b % i == 0) /* i is divisor of both a and b */
            ret = i;
    }
    return ret;
}
```

[Source: Lecture 5, slides 23–24]

Functions, scope, and modules (from lecture 5)

Lecture 5 goes beyond the laboratory and introduces functions: **divide and conquer** — breaking the program into simpler subproblems. Example — linear Diophantine equation ax + by = c: get a, b, c; compute g = gcd(a, b); if c is not a multiple of gcd — no solutions; run the extended Euclidean algorithm; scale x and y by (c/g); print the solution. The Euclidean algorithm relies on gcd(a, b) = gcd(b, a mod b):

```
int gcd(int a, int b) {
    while (b) { /* if a < b, performs swap */
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Functions return no more than one value: for multiple outputs (extended Euclid returns gcd, x and y), the lecture uses global variables — variables declared outside functions: they exist for the entire lifetime of the

program and are accessible from any function. A **module** in C: interface (header file .h) + implementation (auxiliary source/object files). The header euclid.h contains function prototypes and extern declarations of global variables; extern tells the compiler that the variable is defined in another file; compile with one command gcc -g -O0 -Wall diophant.c euclid.c -o diophant.o (both files need to be compiled and linked):

```
/* ensure included only once */
#ifndef EUCLID_H
#define EUCLID_H
/* global variables (declared in euclid.c) */
extern int x, y;
/* compute gcd */
int gcd(int a, int b);
/* compute g = gcd(a, b) and solve ax+by=g */
int ext_euclid(int a, int b);
#endif
```

Scope — the region in which a variable is valid; usually this is the block with the variable declaration. Variables outside functions are global. Lecture example: global int nmax = 20; and a Fibonacci loop in main; if inside main we declare a local int nmax = 25;, it **shadows** the global one — the loop will go to 25 and 25 lines will be printed. **static**: outside a function — the variable/function is visible only inside the file (cannot be extern); inside a function — the variable is local, but it is initialized once when the program starts and is not reinitialized on calls (static int somePersistentVar = 0;). **register**: an explicit request to store the variable in a register; only local variables and arguments; excess/invalid declarations are ignored; you cannot take the address of a register variable — a pointer to it is invalid.

```
int nmax = 20;
/* The main() function */
int main(int argc, char **argv) /* entry point */
{
    int a = 0, b = 1, c, n;
    printf("%3d: %d\n", 1, a);
    printf("%3d: %d\n", 2, b);
    for (n = 3; n <= nmax; n++) {
        c = a + b;
        a = b;
        b = c;
        printf("%3d: %d\n", n, c);
    }
    return 0; /* success */
}
```

[Source: Lecture$_5$, slides 25–43]

Laboratory: string length, do-while and continue

String length (CS50 supplement): string name = get$_{\text{string}}$("Name: "); the string stores H, I, ! and the terminator '\0'. Algorithm: start counting from zero; count each character; stop at the end marker; **the terminator is not counted**. For HI!: n = 0 reads H, n = 1 reads I, n = 2 reads !, n = 3 reaches NUL — the condition is false before the next increment, 3 is printed. An empty string gives 0. Without strlen and pointers.

do-while (optional fragment §6.2.2, file 05$_{\text{textdowhile}}$.c): reads characters except newline, and **also saves the sentinel %** — input % saves one character before the condition becomes false; the condition checks both the input and the capacity (Counter < MaxLength); the text is not a C-string. **continue** (optional fragment §6.4, file 06$_{\text{skipletters}}$.c): reads five characters that are not newline, and skips processing letters; for 'A' continue bypasses printf and goes to the for i++ update; for '1' printf processes the character, then the same update; input A1b2! leaves 1, 2 and !.

[Source: Week-5-Control-Structures, slides 12–14]

3.2. Week 5 Questions — lecture and laboratory questions with model answers

S1. Laboratory survey: what is the role of main in our C programs?

Answer. main — entry point: program execution starts with main. stdio.h declares printf; cs50.h declares get_{double} (and other CS50 input helpers).

S2. How does a block differ from a simple statement and what can be done inside a block?

Answer. A simple statement ends with ';'. Curly braces combine multiple statements into a compound statement (block): it can substitute for a simple statement, compiles as a single unit, variables can be declared inside (they are local to the block), a block can be empty {}, and no ';' is placed after the closing brace. Blocks nest inside each other.

S3. What is considered "true" in a C condition?

Answer. A condition is an expression (or series of expressions), such as n < 3 or x < y || z < y. Any non-zero value ⇒ condition is true, 0 is false. The expression must be numeric (or a pointer); in C89/C90 there is no boolean type, in C99 bool is available via <stdbool.h>. Example: if (str) — string is not null.

S4. What does else if do and what happens if multiple conditions are true?

Answer. else if adds alternative control paths. Conditions are checked in order until the first true one; the inner statement is executed only for it. If multiple conditions are true, only the first branch is executed. The construct is equivalent to nested if.

S5. Which if does else belong to in nested ifs and how to change this?

Answer. Without braces, else associates with the nearest inner if. To associate else with the outer if, the inner if must be enclosed in curly braces: if (x % 4 == 0) { if (x % 2 == 0) y = 2; } else y = 1;.

S6. When is switch more convenient than an if-else ladder and what are its limitations?

Answer. switch tests one integer (or character) variable for equality with a set of case labels — different integer constant expressions; ranges and compound conditions require if. default is the "otherwise" branch. According to the book, switch is more efficient than long if-else ladders.

S7. What happens if you forget break in a case?

Answer. Execution "falls through" to the next case without rechecking the value. In the melody (Example 6.1), omitting break after case 2 caused printing C, then D for input A#. Falling through can also be useful: case 'Y': case 'y': execute the same code for both characters.

S8. Melody: why does the program print note C for character 'A'?

Answer. The program calculates the ASCII code of the character modulo 7: code 'A' is 65; 65 % 7 = 2; case 2 prints C. Slide table: A → 65 → 2 → C.

S9. How does do-while differ from while?

Answer. In do-while the condition is evaluated after each iteration, so the body executes at least once; a semicolon is mandatory at the end: } while (...);. In while the condition is evaluated first — the body may not execute at all.

S10. What does the pair of loops from §6.2.1 print and how do they differ?

Answer. Both print 1 2 3. The while version declares int i = 1; directly before the loop; the for version declares the counter in the header — it is only in scope inside the loop. for gathers initialization, condition, and update in one line.

S11. What do break and continue do and exactly where does execution go?

Answer. break; exits the innermost loop or switch (e.g., early exit from do-while when c != 'y'). continue; skips the rest of the innermost loop body and jumps to its condition; in the lab's for loop, continue with 'A' bypasses printf and lands on i++, then — on the loop condition.

S12. What three expressions are in the for header and what can be omitted?

Answer. Initialization; condition; increment — separated by semicolons. Any of the expressions can be empty; an empty condition is considered true. Expressions can be compound via the comma operator: for (i = 1, j = 1; i <= n; j *= i, i++) — comma has the lowest precedence and is evaluated left to right, this is not a comma between function arguments.

S13. How to manually calculate the length of string HI! without strlen?

Answer. Start counting n from zero; while name[n] != '\0', increment n; stop at the end marker — the terminator is not counted. n = 0 reads H, n = 1 reads I, n = 2 reads !, n = 3 reaches NUL — condition is false, prints 3.

S14. What does a static variable inside a function do and how does it differ from register?

Answer. static inside a function: the variable remains local, but is initialized only once at program startup and is not reinitialized on each call; static outside a function limits visibility to the current file (cannot be extern). register — a request to store the variable in a register: only local variables and arguments; excess declarations are ignored; the address of a register variable cannot be taken.

3.3. Week 5 Tests

T1. What does break do inside a switch?

a) terminates the entire program
b) exits the switch; execution continues after the closing brace
c) goes to default
d) skips the loop condition

Answer: b) exits the switch; execution continues after the closing brace

Explanation: "break leaves the switch" — after printf in case 'B' there is a break, and the program continues after the switch; break does not terminate the program.

T2. How many times is the body of a do-while guaranteed to execute?

a) zero times
b) exactly once
c) at least once
d) depends on the compiler

Answer: c) at least once

Explanation: do-while checks the condition after each iteration ("condition evaluated after each iteration; body executed at least once").

T3. What does for (int i = 1; i <= 3; i++) printf("%d ", i); print?

a) 1 2 3
b) 1 2 3 4
c) 0 1 2 3
d) 3 2 1

Answer: a) 1 2 3

Explanation: comparison of while/for from §6.2.1 — both loops print 1 2 3; the next increment gives 4, and the test becomes false.

T4. What happens if the condition is omitted in the for header?

a) compilation error
b) the body will not execute at all
c) the condition will be considered true
d) the loop will execute exactly once

Answer: c) the condition will be considered true

Explanation: "Expressions can be empty (condition assumed to be "true")" — an empty for condition is always true.

T5. What does continue; do?

a) exits the loop
b) skips the rest of the innermost loop body and jumps to the condition
c) restarts the loop from initialization
d) exits the function

Answer: b) skips the rest of the innermost loop body and jumps to the condition

Explanation: continue; skips the rest of the innermost loop body, jumping to the loop condition; in the gcd example, a non-divisor i is simply skipped.

T6. What is 65 % 7?

a) 1
b) 2
c) 3
d) 9

Answer: b) 2

Explanation: Example 6.1 melody: code 'A' is 65, 65 % 7 = 2 — case 2 prints note C.

T7. Which if does an else without curly braces belong to in nested ifs?

a) to the outermost
b) to the nearest inner
c) to the first in the file
d) to none — this is a compilation error

Answer: b) to the nearest inner

Explanation: slide "Nesting if statements": else associates with the inner if; to associate with the outer, braces are needed.

T8. When is the while loop condition checked?

a) after the body
b) before the body
c) in the middle of the body
d) only on the first iteration

Answer: b) before the body

Explanation: "Condition evaluated first, so body may never be executed" — the while body may never execute.

T9. In C99, the bool type is available through the header…

a) <stdbool.h>
b) <bool.h>
c) <stdio.h>
d) <logic.h>

Answer: a) <stdbool.h>

Explanation: "in C99, bool type available (use stdbool.h)"; in C89/C90 there is no bool type — the condition is just a numeric expression.

T10. Which operator connects multiple expressions in the for header?

a) semicolon
b) comma
c) &&
d) colon

Answer: b) comma

*Explanation: "Compound expressions separated by commas… Comma: operator with lowest precedence, evaluated left-to-right; not same as between function arguments" — for (i = 1, j = 1; i <= n; j *= i, i++).*

3.4. Assignments — glossary for week 5

Term (atama)	Meaning
Block / compound statement	{ … } — combines statements into a single unit; compiled as one whole; creates a local scope; semicolon not needed after }
Branching (selection)	choosing the execution path: if / else if / else and switch
Condition	expression: non-zero value is true, 0 is false; numeric expression or pointer
switch	selection statement by integer (character) variable; content is a block
case label	integer constant label — entry point into a switch block; labels are distinct
default	the "otherwise" branch of a switch — when no case matches
Fall-through	continuing execution to the next case in the absence of break; used to combine case 'Y': case 'y':
while	loop with precondition: condition is evaluated before the body; the body may not execute
for	"counting" loop: initialization; condition; increment in the header; empty condition is true
do-while	post-condition loop: body executes at least once; semicolon at the end
break	exit from the innermost loop or switch
continue	skip the remainder of the inner loop body and jump to the condition (in for — to the update)
Sentinel value	input delimiter character: '#' in the melody, '%' in the do-while example
Scope and Shadowing	scope — region of variable validity (usually the block of declaration; global — outside functions); shadowing — inner declaration with the same name hides the outer one (local nmax = 25 hides global 20)

4. Week 6 — Functions and Structures

Week files: Week-6-Functions-and-Structures.txt (lab "WEEK 6 · CHAPTER 7: From Scratch blocks to a coffee machine") · Lecture$_6$.txt (TashTech Lecture 6 — October 20, 2026)

Week 6 connects Scratch blocks to real C functions: definition, prototype, call, parameters and arguments, return values — and adds structures (typedef struct, fields, dot) on a cool coffee machine model. Lecture 6 reinforces control structures (goto and spaghetti code) and completely covers input-output: standard I/O, printf/scanf formatting, strings, files, and command-line arguments. Below are all the rules, questions, tests, and terms of the week.

4.1. Key Points ("all rules" of week 6)

Function: definition, prototype, call

A function is a named set of instructions. Read the header void print$_{helloworld}$(void): void before the name — no return result; (void) — no inputs; curly braces — body. It's important to distinguish between return value and printed output.

```
void print_hello_world(void)
{
    printf("Hello, world!\n");
}
// calls inside main
print_hello_world();
print_hello_world();
```

Complete program: **the prototype declares the function before main calls it** — ends with ';' and has no body. Find it in the code: Prototype — announce the function; Call — run it now; Definition — describe its work. The program output is Hello, world! twice:

```
#include

void print_hello_world(void);

int main(void)
{
    print_hello_world();
    print_hello_world();
    return 0;
}

void print_hello_world(void)
{
    printf("Hello, world!\n");
}
```

If you change the message inside the function, both calls use the modified definition: reuse is common behavior, not just less printing.

[Source: Week-6-Functions-and-Structures, slides 5–6]

Parameters, arguments, and return value

One function can use another. **Parameter** — the name of the input inside the function (count); **argument** — the value passed by the call (3). count is local to print$_{meows}$; on each call, the value is copied:

```
void print_meow(void)
{
    printf("meow\n");
}

void print_meows(int count)
{
    for (int i = 0; i < count; i++)
```

```
    {
        print_meow();
    }
}

// calls inside main
print_meows(1);
print_meows(3);
```

The return value goes back to the caller. The textbook example IntegerRoot — integer square root: int — result type; number — input parameter; caller — the function that makes the call (here main). IntegerRoot returns a number; **the caller prints it** via printf:

```
int IntegerRoot(int number);

// calls inside main
int first = IntegerRoot(27);
int second = IntegerRoot(16);
printf("%d %d\n", first, second);
```

For 27: $5 \times 5 = 25 \leq 27$; $6 \times 6 = 36 > 27$ — answer 5. For 16 — 4. The program prints **5 4**.

[Source: Week-6-Functions-and-Structures, slides 7–8]

Structures: typedef struct and field access

Structure type definition is placed above main: struct { … } groups related fields into one type; int $water_{ml}$; declares one integer field; typedef gives the type a convenient name; Stock — the type name after the closing brace; the entire declaration ends with ';'. **Field — one named value in a record.**

```
typedef struct
{
    int water_ml;
    int milk_ml;
    int beans_g;
    int paper_cups;
} Stock;
```

Creating a variable and working with fields: values go in field order — water, milk, beans, cups; access to one field — via dot. Stock (type, capital) and stock (variable, lowercase) — different names:

```
Stock stock = {500, 200, 40, 3};
printf("%d\n", stock.water_ml);
stock.water_ml = 450;
printf("%d\n", stock.water_ml);
```

Field	Value after initialization
$water_{ml}$	500
$milk_{ml}$	200
$beans_{g}$	40
$paper_{cups}$	3

stock after initialization; program output: 500, then 450 — only $water_{ml}$ field changes [Source: Week-6-Functions-and-Structures, slide 10]

[Source: Week-6-Functions-and-Structures, slides 9–10]

Copy of a record: function returns a separate Stock

The helper receives a **copy** of the Stock parameter: modifies the copy and returns it; the caller decides where to save the result:

```
Stock stock_after_water_use(
    Stock stock, int amount_ml)
{
    if (amount_ml >= 0 &&
        amount_ml <= stock.water_ml)
    {
        stock.water_ml -= amount_ml;
    }
    return stock;
}

// Caller · stock starts with 500 ml
Stock preview =
    stock_after_water_use(stock, 100);
printf("%d\n", stock.water_ml);
printf("%d\n", preview.water_ml);
stock =
    stock_after_water_use(stock, 50);
printf("%d\n", stock.water_ml);
```

Moment	$stock.water_{ml}$	$preview.water_{ml}$
After first call	500	400
After saving to stock	450	400

First call returns the modified copy, original remains 500; next call for 50 ml starts from unchanged 500 and saves 450 to stock [Source: Week-6-Functions-and-Structures, slide 11]

[Source: Week-6-Functions-and-Structures, slide 11]

Coffee machine model: $can_{makecoffee}$, $stock_{aftercoffee}$ and Cup

Class model: machine has water 500 / milk 200 / beans 40 / cups 3; one latte requires 100 / 50 / 10 / 1. **Recipe is also a type-structure (Recipe); variable latte stores amounts for one drink; one accepted order uses one cup.** Subtract each resource: 400 ml water, 150 ml milk, 30 g beans, 2 cups.

Solution function: bool — result true/false; >= — "not less than this amount"; && — each condition must be satisfied; return — passes the result to the caller. "Exactly this amount" is accepted; one missing cup is enough to stop the order:

```
bool can_make_coffee(
    Stock stock, Recipe recipe)
{
    return
        stock.water_ml >= recipe.water_ml &&
        stock.milk_ml >= recipe.milk_ml &&
        stock.beans_g >= recipe.beans_g &&
        stock.paper_cups >= recipe.paper_cups;
}
```

Stock for this latte	Can make?
500 / 200 / 40 / 3	true
100 / 50 / 10 / 1	true
500 / 200 / 40 / 0	false

Numbers are water/milk/beans/cups [Source: Week-6-Functions-and-Structures, slide 13]

Calculating next Stock: if coffee is possible — subtract all four resources and return a copy; -= subtracts and saves the result; return stock; sends the updated copy back. Compare two calls:

```
Stock stock_after_coffee(
    Stock stock, Recipe recipe)
{
    if (can_make_coffee(stock, recipe))
    {
        stock.water_ml -= recipe.water_ml;
        stock.milk_ml -= recipe.milk_ml;
        stock.beans_g -= recipe.beans_g;
        stock.paper_cups -= recipe.paper_cups;
    }
    return stock;
}

stock_after_coffee(stock, latte);
// stock is still unchanged

stock =
    stock_after_coffee(stock, latte);
// stock now keeps the returned record
```

Caller's Stock	Water / milk / beans / cups
Before order	500 / 200 / 40 / 3
After saving result	400 / 150 / 30 / 2

First call discards the returned copy, second one saves it [Source: Week-6-Functions-and-Structures, slide 14]

Order block: $\text{print}_{\text{stock}}$ and $\text{print}_{\text{cup}}$ show data; $\text{create}_{\text{cup}}$ builds a drink record (Cup); false means "no foam". If there's no stock ($\text{stock.paper}_{\text{cups}} = 0$), the if branch doesn't perform any assignments — all current stock fields and the previous Cup remain unchanged:

```
print_stock(stock);
if (can_make_coffee(stock, latte))
{
    stock =
        stock_after_coffee(stock, latte);
    cup = create_cup(latte, false);
}
print_stock(stock);
print_cup(cup);
```

[Source: Week-6-Functions-and-Structures, slides 12–15]

goto, spaghetti code and error handling

goto allows unconditional jump to any place in the code **within the same function**; the place is marked with a **label** — a named point having the same form as a variable name, followed by ':'.

```
start:
{
    if (cond)
        goto outside;
    /* some code */
    goto start;
}
outside:
/* outside block */
```

Dijkstra, "Go To Statement Considered Harmful" (Communications of the ACM 11(3), 1968): excessive goto creates **spaghetti code** — code becomes harder to read and debug; any code with goto can be rewritten without it. But in C, goto has a legitimate use: in C++ and Java there's an exception mechanism for error recovery, while in C **goto is a convenient way to exit nested blocks** (jumping to error: label skips both

blocks).

[Source: Lecture$_6$, slides 5–7]

Input-output: putchar/getchar, printf and its formats, scanf

Input-output facilities are provided by the standard library **<stdio.h>**, not the language itself. **Text stream** — a series of strings ending with '\n' (the library itself converts '\r\n' to '\n'); **binary stream** — a series of raw bytes; library streams are buffered.

int putchar(int) puts character c to standard output; returns the printed character or EOF on error. **int getchar()** returns the next character from standard input or EOF on error. Classic example — case conversion (c = c - 'A' + 'a'; — possible due to char's integer conversion):

```
int main()
{
    char c;
    while ((c = getchar()) != EOF)
    {
        if (c >= 'A' && c <= 'Z')
            c = c - 'A' + 'a';
        putchar(c);
    }
    return 0;
}
```

To read from a file instead of standard input, the '<' operator is used (*nix): ./a.out < file.txt — this is an OS feature, not a language feature.

int printf(char format[], arg1, arg2, ...) — formatted output: takes variable number of arguments, returns number of printed characters. Format contains literal text and specifiers starting with %: **%[flags][width][.precision][length]<type>**.

Type	Meaning	Example
d, i	integer	printf("%d",10); / *prints 10* /
u	unsigned integer	printf("%u",10); / *prints 10* /
x, X	integer (hex)	printf("%x",10); / *prints 0xa* /
c	character	printf("%c",'A'); / *prints A* /
s	string	printf("%s","hello"); / *prints hello* /
f	float	printf("%f",2.3); / *prints 2.3* /
e, E	float (exp.)	1e3, 1.2E3, 1E-3
%	literal %	printf("%d %%",10); / *prints 10%* /

printf type specifiers [Source: Lecture$_6$, slide 12; Lecture$_{71}$, slide 3]

Modifier	Meaning
h	interpret as short; use with i, d, o, u, x
l	interpret as long; use with i, d, o, u, x
L	interpret as double; use with e, f, g

Length modifiers [Source: Lecture$_6$, slide 16]

Call	Output
printf("%d",10)	"10"

Call	Output
printf("%4d",10)	"bb10" (b — space: width)
printf("%7s","hello")	"bbhello"
printf("%d,%+d,%+d",10,-10)	"10,+10,-10" (flag +)
printf("%04d",10)	«0010» (flag 0)
printf("%-7s","hello")	«hellobb» (flag -: left alignment)
printf("%.2f,%.0f",1.141,1.141)	«1.14,1» (precision)
printf("%.2e,%.0e",1.141,100.00)	«1.14e+00,1e+02»
printf("%.4s","hello")	«hell»
printf("%.1s","hello")	«h»

width, flags and precision on examples [Source: Lecture$_6$, slides 13–15]

int scanf(char *format, ...) — input analog of printf: reads characters from standard input, interpreting them according to the format specification (the same as printf's); accepts a variable number of arguments; when reading multiple items, each is considered separated by whitespace characters; **scanf ignores whitespace characters**; returns the number of items read or EOF. Important: **arguments must be addresses of variables (pointers)**. Correspondence examples:

printf	scanf
printf("%d",x)	scanf("%d",&x)
printf("%f",f)	scanf("%f",&f)
printf("%s",str)	scanf("%s",str) — & not needed
printf("%s",str)	scanf("%20s",str) — & not needed
printf("%s %s",fname,lname)	scanf("%20s %20s",fname,lname)

printf ↔ scanf [Source: Lecture$_6$, slide 20]

Formatted data can be written to and read from character arrays: **int sprintf(char string[], char format[], arg1, arg2)** — like printf, but output is written to string (size not checked); **int sscanf(char str[], char format[], arg1, arg2)** — like scanf, but input is read from str; both return the number of items written/read or a negative value on error.

[Source: Lecture$_6$, slides 8–21]

Strings as arrays, strcmp, files and command line

Strings are represented as character arrays; C does not limit string length — the end of a string is marked by a zero byte: "hello" is the array {'h','e','l','l','\0'}. Declarations:

```
char str[] = "hello";   /* compiler takes care of size */
char str[10] = "hello"; /* make sure the array is large enough */
char str[] = {'h','e','l','l',0};
```

Note: to include " in a string, use \". For string comparison, the header **<string.h>** provides **int strcmp(char s[], char t[])** — dictionary order comparison (lowercase letters come after uppercase): returns a value <0 if s comes before t; 0 if strings are equal; >0 if s comes after t; strcmp is case-sensitive. Examples: strcmp("A","a") → <0; strcmp("IRONMAN","BATMAN") → >0; strcmp("aA","aA") → ==0; strcmp("aA","a") → >0.

Files: FILE *fopen(char name[], char mode[]) opens a stream; modes include "r" (read), "w" (write), "a" (append), among others; "b" is added for binary files; fopen returns a pointer to the stream or NULL. int*

fclose(FILE fp) closes the stream (releases OS resources); at program termination, fclose is called automatically for all open files. Standard input and output are also FILE *(stdin, stdout); stderr is a separate standard error output stream. Character by character: int getc(FILE* fp) (getchar is getc(stdin): #define getchar() getc(stdin)); int putc(int c, FILE *fp) (putchar is putc(c, stdout)); line by line: char* fgets(char line[], int maxlen, FILE *fp) reads one line up to maxlen characters (including newline), returns a pointer to the array or NULL at end of stream; int fputs(char line[], FILE* fp) writes a string, returns 0 on success or EOF; int fscanf(FILE *fp, char format[], ...) — like scanf/sscanf, but from stream fp.

Command line arguments: main can accept parameters filled when the program is started: **int main(int argc, char *argv[])**; argc is the argument counter; argv[] is an array of pointers to each argument; arguments include the program name: ./cat a.txt b.txt gives argc = 3, argv[0] = "cat", argv[1] = "a.txt", argv[2] = "b.txt"; ./cat gives argc = 1, argv[0] = "cat".

[Source: Lecture$_6$, slides 17–27]

4.2. Week 6 Questions — lecture and lab questions with model answers

S1. What do void before the function name and (void) in parentheses mean?

Answer. void before the name — no return result; (void) — no inputs. In void print$_{helloworld}$(void) both voids are in their places; braces enclose the body. Return value needs to be distinguished from printed output.

S2. How does a prototype differ from a call and a definition?

Answer. A prototype declares a function before main: void print$_{helloworld}$(void); — ends with ';' and has no body. A call — print$_{helloworld}$(); — executes the function now. A definition — the body with curly braces — describes its operation. In one program: prototype before main, calls inside main, definition after.

S3. Lecture recap question: what does for (int i = 1; i <= 3; i++) printf("%d ", i); print?

Answer. 1 2 3 — the counter goes through 1, 2, 3; when i = 4, the condition i <= 3 is false. The same result was given by the while/for comparison from week 5.

S4. Recap question: what is streak after int streak = 2; int practised = 0; if (practised == 1) streak++; else streak = 0;?

Answer. streak = 0: practised == 1 is false (practised equals 0), the else branch is executed, and streak is reset to zero. Any non-zero value is true, 0 is false.

S5. Recap question: without the first break, what is score? int score = 0; char reaction = 'W'; switch (reaction) { case 'W': score += 2; case 'L': score -= 1; break; }

Answer. score = 1. The character 'W' matches case 'W': score += 2 gives 2; no break — execution falls through to case 'L' without rechecking: score -= 1 gives 1; break terminates the switch.

S6. How does a parameter differ from an argument?

Answer. A parameter is the input name inside a function: count in void print$_{meows}$(int count); this name is local to the function. An argument is the value passed by the call: 3 in print$_{meows}$(3). On each call, the value is copied into the parameter.

S7. What does the pair of calls IntegerRoot(27) and IntegerRoot(16) print?

Answer. 5 4. For 27: 5 × 5 = 25 ≤ 27, and 6 × 6 = 36 > 27, so the integer root is 5; for 16 — 4. IntegerRoot returns a number, and the caller prints it via printf("%d %d\n", first, second);.

S8. What is a field and how does Stock differ from stock?

Answer. A field is one named value in a struct (water$_{ml}$, milk$_{ml}$, beans$_{g}$, paper$_{cups}$). Stock is the type name introduced by typedef after the closing brace; stock is a variable of that type. Case matters: Stock and stock are

different names.

S9. Why after Stock preview = stock$_{afterwateruse}$(stock, 100); is the water in stock still 500, but in preview — 400?

Answer. The stock parameter inside the function is a copy: the helper subtracts 100 from the copy and returns the modified copy; the original in main remains 500. The caller chooses where to save the result: preview gets 400. After stock = stock$_{afterwateruse}$(stock, 50); — a new call starts from the unchanged 500 and saves 450 in stock.

S10. What does the assignment stock = stock$_{aftercoffee}$(stock, latte); change, and what happens without it?

Answer. With the assignment, stock saves the returned struct: 400 / 150 / 30 / 2. Without assignment (just stock$_{aftercoffee}$(stock, latte);) the returned copy is discarded — stock doesn't change: "stock is still unchanged".

S11. Why when stock.paper$_{cups}$ = 0 is the order not fulfilled and nothing changes?

Answer. can$_{makecoffee}$ requires all four resources to be no less than required (&& — each condition must be met): one missing cup is enough to return false. The if branch performs no assignments — all fields of stock and the previous cup (latte) remain unchanged.

S12. What is dangerous about goto and when is it justified in C?

Answer. Excessive goto creates spaghetti code (Dijkstra, "Go To Statement Considered Harmful", CACM 11(3), 1968): code is harder to read and debug; any code with goto can be rewritten without it. In C goto is justified as a convenient way to exit nested blocks during error handling — in C++/Java there are exceptions for this, but not in C.

S13. Why does scanf require & and when is it not needed?

Answer. scanf arguments must be variable addresses (pointers): scanf("%d", &x) writes the read number to the address of x. For character arrays (strings), the array name is already an address: scanf("%s", str) — without &; similarly scanf("%20s", str) and scanf("%20s %20s", fname, lname). scanf ignores whitespace characters and returns the number of items read or EOF.

S14. What do argc and argv store?

Answer. int main(int argc, char *argv[]): argc is the number of arguments; argv[] is an array of pointers to each argument; arguments include the program name. Example: ./cat a.txt b.txt gives argc = 3, argv[0] = "cat", argv[1] = "a.txt", argv[2] = "b.txt"; a single ./cat gives argc = 1.

4.3. Week 6 Tests

T1. What is a function prototype?

a) function body with curly braces

b) function declaration ending with ';' and without a body

c) function call from main

d) variable inside a function

Answer: b) function declaration ending with ';' and without a body

Explanation: "A prototype declares a function before main calls it. It ends with ; and has no body" — a prototype declares, a call executes, a definition describes the work.

T2. How many inputs and return values does void print$_{meow}$(void) have?

a) 1 input, 1 value

b) no inputs, no return value

c) no inputs, 1 value

d) 1 input, no value

Answer: b) no inputs, no return value

Explanation: void before the name — no return result; (void) — no inputs.

T3. What does IntegerRoot(27) return?

a) 4

b) 5

c) 6

d) 27

Answer: b) 5

Explanation: $5 \times 5 = 25 \le 27$, and $6 \times 6 = 36 > 27$ — the integer square root of 27 is 5; a pair of calls prints 5 4.

T4. How to access the $water_{ml}$ field of the stock variable?

a) $stock.water_{ml}$

b) $Stock.water_{ml}$

c) $stock->water_{ml}$

d) $stock:water_{ml}$

Answer: a) $stock.water_{ml}$

Explanation: accessing a field is done via a dot: $stock.water_{ml} = 450$; (Stock is the type name, not a variable).

T5. In typedef struct { … } Stock; the name Stock is…

a) variable

b) type name introduced by typedef

c) structure field

d) function

Answer: b) type name introduced by typedef

Explanation: typedef gives a type a convenient name; Stock comes after the closing brace — it's a type name, not a variable and not a struct tag.

T6. After Stock preview = $stock_{afterwateruse}$(stock, 100); what is $stock.water_{ml}$?

a) 400

b) 500

c) 450

d) 0

Answer: b) 500

Explanation: the stock parameter in the function is a copy; the first call returns the modified copy ($preview.water_{ml} = 400$), the original remains 500.

T7. What will printf("%4d", 10); print?

a) "10"

b) "bb10" (b — space)

c) "0010"

d) "10bb"

Answer: b) "bb10" (b — space)

Explanation: width 4 pads the number with spaces on the left; "0010" is given by flag 0 (%04d), "hellobb" is by flag minus (%-7s).

T8. What will printf("%.4s", "hello"); print?

a) hello
b) hell
c) h
d) ello

Answer: b) hell

Explanation: precision for strings truncates the output to 4 characters; %.1s will give "h", and %.2f gives two decimal places.

T9. How to correctly read an integer into variable x?

a) scanf("%d", x);
b) scanf("%d", &x);
c) scanf("%d", *x);
d) scanf(%d, &x);

Answer: b) scanf("%d", &x);

Explanation: scanf arguments must be addresses of variables (pointers); for a char array, the name is already an address: scanf("%s", str).

T10. What does the "r" mode of the fopen function do?

a) opens the file only for writing
b) opens the file only for reading
c) opens the file for appending
d) makes the file binary

Answer: b) opens the file only for reading

Explanation: "r" — read only, "w" — write only, "a" — append; "b" is added for binary files; fopen returns NULL if the stream doesn't exist.

4.4. Assignments — glossary of week 6

Term (atama)	Meaning
Function	a named set of instructions; void $print_{helloworld}$(void) — definition, $print_{helloworld}$(); — call
Prototype	function declaration before main: type name(parameters); — ends with ';' and has no body
Definition and call	definition — the body of the function in braces ("describe its work"); call — running the function by name ("run it now"): $print_{helloworld}$();
Parameter	the name of an input inside the function: count in void $print_{meows}$(int count)
Argument	the value passed by a call: 3 in $print_{meows}$(3)
Return value	the result that the function sends to the caller: int IntegerRoot(int number)
Structure	a type that groups related fields into one record: struct { … }
Field	one named value in a record: $water_{ml}$, $milk_{ml}$, $beans_{g}$, $paper_{cups}$
typedef	gives a structural type a convenient name: typedef struct { … } Stock; a variable of this type is a record: Stock stock = {500, 200, 40, 3}; access fields via a dot
Label	a named point in code for goto: name + ':' (start:, outside:, error:)

Term (atama)	Meaning
Spaghetti code	unreadable, hard-to-debug code from excessive goto (Dijkstra, 1968)
Text stream	a series of lines ending with '\n'; conversion of '\r\n' is handled by the library; streams are buffered
Format specifier	%[flags][width][.precision][length]<type>: %d, %4d, %04d, %.4s, %e, %%
Command line arguments	int main(int argc, char *argv[]): argc — counter, argv — array of pointers to arguments, including the program name

5. Week 7 — Pointers, Strings, and Recursion

Files of the week: Lecture$_{71}$.txt (TashTech Lecture 7 — October 27, 2026: Pointers and Memory Addresses · Arrays and Pointer Arithmetic · Strings · Searching and Sorting) · Recursion-Lecture.txt (WEEK 7 · CHAPTER 8: Recursion)

Week 7 — double topic. Lecture 7 covers pointers: variable addresses, physical and virtual memory, dereferencing, functions with multiple outputs, arrays and pointer arithmetic, strings and string.h functions, as well as basic search and sorting algorithms (linear search, insertion sort, quicksort, binary search). Chapter 8 lab teaches recursion: base case, smaller problem, progress; tracing factorial through the call stack, fibonacci, and Tower of Hanoi. Below are all the rules, questions, tests, and terms of the week.

5.1. Key points ("all rules" of week 7)

Pointer: address (&) and dereferencing (*)

A pointer is the address of a variable in memory; the address can be used to access and modify the variable from anywhere — this is extremely useful for data structures (and is notoriously confusing in code). **Every variable in memory has an address.** Variables without addresses: register variables; constants/literals/preprocessor defines; expressions (if the result is not a variable). The address is taken by the operator **&**; the address of a variable of type t has type **t ***:

```
int n = 4;
double pi = 3.14159;
int *pn = &n;      /* address of integer n */
double *ppi = π /* address of double pi */
```

Dereferencing/indirection — the * operator: access to a variable by address; a dereferenced pointer behaves like a regular variable. **Null pointer** — 0 (NULL): a pointer that doesn't reference anything:

```
/* prints "pi = 3.14159\n" */
printf("pi = %g\n", *ppi);

/* pi now equals 7.14159 */
*ppi = *ppi + *pn;
```

Pointers can be explicitly cast to any pointer type (implicit casting to/from void * is also possible); a dereferenced pointer has a new type regardless of the actual data; this can cause segmentation faults and other hard-to-catch errors:

```
ppi = (double *)pn; /* pn originally of type (int *) */
```

[Source: Lecture$_{71}$, slides 5, 9–11]

Memory: physical and virtual, stack and heap

Physical memory — resources where data is actually stored: cache, RAM, hard disk, removable media; they have different capacities and access speeds; memory management is one of the main functions of the OS (it moves data during execution); in embedded processors, memory can be extremely limited. **Virtual memory** — an OS abstraction: address space available to code (on a 32-bit OS < 4 GB: usually 2 GB for Windows, 3–4 GB for Linux); virtual memory is mapped to different parts of physical memory.

Used parts of virtual memory: stack and heap. Stack (stack) — where declared variables are placed; heap (heap) — where dynamic memory is placed.

Pointers in functions: multiple outputs and swap

Functions return no more than one value. The extended Euclidean algorithm returns gcd(a, b), x and y (ax + by = gcd) — instead of global variables, **pointers can be passed for multiple outputs**:

```
int ext_euclid(int a, int b, int *x, int *y);

int x, y, g;
/* assume a, b declared previously */
g = ext_euclid(a, b, &x;, &y;);
```

For a function to be able to **modify variables of the caller**, pointers to them are passed. Classic swap:

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int a = 5, b = 7;
swap(&a;, &b;);
/* now, a = 7, b = 5 */
```

The reverse pitfall — **variable going out of scope**: returning a pointer to a local array is invalid, the pointer becomes invalid after the function's stack frame disappears (compilers usually warn):

```
#include

char * get_message() {
    char msg[] = "Aren't pointers fun?";
    return msg;
}

int main(void) {
    char *string = get_message();
    puts(string);
    return 0;
}
```

[Source: Lecture$_{71}$, slides 12–14]

Arrays, sizeof and address arithmetic

Primitive arrays are implemented in C as a pointer to a block of continuous memory. arr behaves like a pointer to element 0 — but, unlike a pointer, it cannot be reassigned:

```
int arr[8];
int a = arr[0];
int *pa = arr;  /* same as int *pa = &arr;[0]; */
```

sizeof(): for primitive types/variables — the size of the type in bytes; for primitive arrays — the size of the entire array in bytes. Array length — is a macro:

```
int s = sizeof(char);  /* == 1 */
```

```
double f;              /* sizeof(f) == 8  (64-bit OS) */
int arr[8];            /* sizeof(arr) == 32 (64-bit OS) */
long arr[5];           /* sizeof(arr) == 40 (64-bit OS) */

/* needs to be on one line when implemented */
#define array_length(arr) (sizeof(arr) == 0 ? \
    0 : sizeof(arr) / sizeof((arr)[0]))
```

Address arithmetic: a pointer is not an int, but an integer can be added/subtracted to it: **pa + i points to arr[i]**; the address value increases by i multiplied by the size of the data type. If arr[0] has address 100 (int), then arr[3] has address 112 (3 × 4 bytes). Slide question: if char *pc = (char *)pa;, then (int *)(pc + i) == pa + 3 when **i = 12** (3 int = 12 bytes per char).

[Source: Lecture$_{7}$$_{1}$, slides 15–17]

Strings and the string.h library

Strings are stored as **arrays of characters terminated by a null byte** (last character == '\0'). Work with strings as with arrays:

```
char str[] = "This is a string.";
char *pc = str;

*(pc+10) = 'S';
puts(str); /* prints "This is a String." */
```

Standard header **<string.h>** functions: copying — strcpy(), strncpy(); comparison — strcmp(), strncmp(); length — strlen(); concatenation — strcat(), strncat(); character search — strchr(), strrchr(). Exact slide signatures:

- char *strcpy(strto, strfrom); — copies strfrom to strto
- char *strncpy(strto, strfrom, n); — copies n characters from strfrom to strto
- int strcmp(str1, str2); — compares str1 and str2: 0 if equal; positive if str1 > str2; negative if str1 < str2
- int strncmp(str1, str2, n); — compares the first n characters of str1 and str2
- int strlen(str); — returns the length of str
- char *strcat(strto, strfrom); — appends strfrom to the end of strto
- char *strncat(strto, strfrom, n); — appends n characters from strfrom to the end of strto
- char *strchr(str, c); — searches for character c in str: pointer to the first occurrence or NULL
- char *strrchr(str, c); — pointer to the last occurrence of c or NULL

[Source: Lecture$_{7}$$_{1}$, slides 18–20]

Search and sort: linear, insertion, quick, binary

Basic algorithms make excellent use of pointers; these are just a few examples, not an algorithms course; complexity is described in **Big-O notation**. Linear search returns a pointer to the found element or NULL:

```
int arr[100]; /* array to search */

int * linear_search(int val) {
    int *parr, *parrend = arr + array_length(arr);
    for (parr = arr; parr < parrend; parr++) {
        if (*parr == val)
            return parr;
    }
    return NULL;
}
```

Simple insertion sort — O(n^2): go through the array until an element is found out of place; insert it in the correct position; repeat until the end of the array. Split into two functions:

```
/* move previous elements down until insertion point reached */
void shift_element(unsigned int i) {
    int ivalue;
    /* guard against going outside array */
    for (ivalue = arr[i]; i && arr[i-1] > ivalue; i--)
        arr[i] = arr[i-1]; /* move element down */
    arr[i] = ivalue;       /* insert element */
}

/* iterate until out-of-order element found;
   shift the element, and continue iterating */
void insertion_sort(void) {
    unsigned int i, len = array_length(arr);
    for (i = 1; i < len; i++)
        if (arr[i] < arr[i-1])
            shift_element(i);
}
```

Fast sorts (shellsort, mergesort, quicksort…): quicksort — O(n log n) on average, O(n^2) in the worst case; choose a pivot element; move smaller elements to one side, larger ones to the other; sort the sides separately (**recursive algorithm**); implemented in the standard library as qsort() in stdlib.h. Implementation from K&R:

```
void quick_sort(unsigned int left, unsigned int right) {
    unsigned int i, mid;
    int pivot;
    if (left >= right)
        return; /* nothing to sort */
    /* pivot is midpoint; move to left side */
    swap(arr + left, arr + (left + right) / 2);
    pivot = arr[mid = left];
    /* separate into side < pivot (left+1 to mid)
       and side >= pivot (mid+1 to right) */
    for (i = left + 1; i <= right; i++)
        if (arr[i] < pivot)
            swap(arr + ++mid, arr + i);
    /* restore pivot position */
    swap(arr + left, arr + mid);
    /* sort two sides */
    if (mid > left)
        quick_sort(left, mid - 1);
    if (mid < right)
        quick_sort(mid + 1, right);
}

quick_sort(0, array_length(arr) - 1);
```

Discussion of quicksort: in its current form, it's unstable (equal elements may change places); sorts in place — especially valuable with limited memory; choice of pivot affects performance (can be random); divide and conquer, easily parallelizable; recursive — in the worst case, it can overflow the stack on a large array.

Binary search — O(log n) on average and in the worst case: the list must be sorted; compare with the middle element of the current segment — the new segment is guaranteed to be half the size (divide and conquer!). Requires **random access** to the array's memory: on sequential data (hard disk), forward-backward search is wasteful — sometimes linear is better. Implemented in stdlib.h as bsearch():

```
int * binary_search(int val) {
    unsigned int L = 0, R = array_length(arr), M;
    while (L < R) {
```

```
        M = (L + R - 1) / 2;
        if (val == arr[M])
            return arr + M;  /* found */
        else if (val < arr[M])
            R = M;           /* in first half */
        else
            L = M + 1;       /* in second half */
    }
    return NULL; /* not found */
}
```

[Source: Lecture$_{71}$, slides 21–32]

Recursion: base case, smaller problem, progress

Recursion — a function calls itself for a smaller problem. A cool analogy is "queue": how many people are there from you to the end of the queue, including you? Ask the next person and wait; the last one answers "only me — 1"; the answers return, each adds themselves: 1 + 1 = 2, 2 + 1 = 3, 3 + 1 = 4. The same method, smaller problem, until one case is solved directly.

Function call **suspends the caller**: main calls double$_{value}$(3) and waits; the called function has its own x = 3; it returns 6; main continues with result = 6. (Returning 6 and printing 6 are different things.)

```
int double_value(int x)
{
    return 2 * x;
}
// Inside main:
int result = double_value(3);
```

First recursive function — countdown:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);
}
```

Base case: at 0 — stop this call. **Recursive call**: same function, smaller input. Tracing count$_{down}$(3): prints 3, 2, 1 (each number on its own line); count$_{down}$(0) returns without printing — 0 is reached but not printed.

Three ingredients of recursion: (1) base case — answer without new call (n == 0); (2) smaller problem — same function on simpler input (count$_{down}$(n - 1)); (3) progress — each step moves toward the base case (3, 2, 1, 0).

Before or after the recursive call? Printing before the call — values appear on the way down; printing after — printing waits for the return of the smaller call:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);
}

void count_up(int n)
{
    if (n == 0)
        return;
    count_up(n - 1);
```

```
    printf("%i\n", n);
}
```

	$count_{down}(3)$	$count_{up}(3)$
Output	3 2 1	1 2 3
Printing	before diving in — on the way down	waits for return of smaller call

The order of printing follows the order of execution [Source: Recursion-Lecture, slides 14–15]

The base case must be reachable: having a base case in the code doesn't guarantee termination. A recursive call with the same n prints 3, 3, 3, 3… — the input never reaches 0; the fix is $count_{down}(n - 1)$. Negative input for this simple function would still break it — the input contract (n >= 0) is important; unbounded recursion can exhaust the stack:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);   /* 3, 3, 3, 3, ... — input doesn't reach 0 */
}

/* Repair */
```

Trace-check $sum_{to}(n)$ — sum of integers from 1 to n ($sum_{to}(0) = 0$): the blank should contain n - 1; $sum_{to}(3) = 3 + 2 + 1 + 0 = 6$; in the frame n = 3, the pending work is "add 3 to the smaller answer":

```
int sum_to(int n)
{
    if (n == 0)
        return 0;
    return n + sum_to(n - 1);
}
```

[Source: Recursion-Lecture, slides 2–8, 14–17]

Factorial, call stack, fibonacci and Hanoi

Factorial: 3! = 3 × 2 × 1 = 6; the part 2 × 1 is 2!, so 3! = 3 × 2! — "the smaller answer inside". Base case: **0! = 1** (if the base returned 0, every product would become zero):

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    int smaller = factorial(n - 1);
    return n * smaller;
}
```

Each call has its own stack frame: the frame (stack frame, activation record) stores what an incomplete call needs: its own parameters/locals and the continuation point. Frame for factorial(3): n = 3; waiting for factorial(2); next action is return 3 * smaller. The new call gets its own n; the caller saves its own incomplete work.

Tracing factorial(3): main waits for result; factorial(3) calls factorial(2) and waits to multiply by 3; factorial(2) → factorial(1) → factorial(0). **Only the top call is active.** The base case factorial(0) returns 1 — but this isn't the final result: reaching the base case doesn't return 6 to main. Returns remove completed frames: factorial(1) gets smaller = 1, computes 1 × 1 = 1; factorial(2): 2 × 1 = 2; factorial(3): 3 × 2 = 6; main continues with result = 6. Order of calls: 3, 2, 1, 0; **returns: 1, 1, 2, 6**.

	Recursive factorial	Iterative factorial
Repeated step	function calls itself	loop repeats the body
What awaits	one incomplete call per level	no recursive calls
State in this example	each call stores its own n	counter and accumulated product
Result for 3	6	6

Recursion and loop [Source: Recursion-Lecture, slide 18]

Don't claim that recursion is always faster or better: for factorial, the loop and recursion give the same result with a similar number of multiplications; but for self-similar tasks (traversing nested structures), the recursive form is often simpler to express.

Fibonacci: 0, 1, 1, 2, 3, 5, 8, … — each next value is the sum of the two preceding ones. **Two base cases** (the first two values are known directly: F(0) = 0, F(1) = 1); two smaller answers — first the left call completely finishes, then the right one starts, then they are added together:

```
int fibonacci(int n)
{
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    int left = fibonacci(n - 1);
    int right = fibonacci(n - 2);
    return left + right;
}
```

fibonacci(4) returns 3. Behind fibonacci(4) there are nine calls (read left to right in explicit code order: 4, 3, 2, 1, 0, 1, 2, 1, 0): fibonacci(3) → 2; fibonacci(2) → 1 (twice!); fibonacci(1) → 1 (three times); fibonacci(0) → 0 (twice). **fibonacci(2) appears twice — the same answer is computed again**; only one call executes at any moment; this simple version repeats work.

- fibonacci(4) → returns 3
- fibonacci(3) → returns 2
- fibonacci(2) → returns 1 (and again 1 — second node)
- fibonacci(1) → returns 1 (three calls)
- fibonacci(0) → returns 0 (two calls)

Tower of Hanoi (Book Exercise 8.5): move all three disks from A to C, using B. Rules: (1) move one disk at a time; (2) move only the top disk of a rod; (3) never place a larger disk on a smaller one. Three recursive steps: (1) move 2 disks from A to B, using C — 3 single-disk moves; (2) move disk 3 from A to C — 1 move; (3) move 2 disks from B to C, using A — 3 moves. Total **3 + 1 + 3 = 7 moves**. Base case: no disks — no moves (n == 0). The exact sequence of seven moves: disk 1 A→C; disk 2 A→B; disk 1 C→B; disk 3 A→C; disk 1 B→A; disk 2 B→C; disk 1 A→C.

[Source: Recursion-Lecture, slides 7–13, 19–22]

5.2. Week 7 questions — lecture and lab questions with model answers

S1. What is a pointer and what is a null pointer?

Answer. A pointer is the address of a variable in memory; the address allows accessing and modifying the variable from anywhere (especially useful for data structures). A null pointer is 0 (NULL): a pointer that doesn't reference anything.

S2. What do the & and * operators do?

Answer. & — is the address-of operator: the address of a variable of type t has type t *(int* pn = &n;). — *is the dereference/indirection operator: access to the value at an address (printf("pi = %g\n",* ppi); or *ppi* = ppi + *pn;). A dereferenced pointer behaves like a regular variable.

S3. How to return multiple values from a function?

Answer. Pass pointers for additional outputs: int ext_{euclid}(int a, int b, int *x, int* y); the call g = ext_{euclid}(a, b, &x, &y); writes x and y to the passed addresses. Functions can return no more than one value — pointers extend this limit.

S4. What's wrong with a function that returns the address of a local array ($get_{message}$)?

Answer. char msg[] lives on the stack frame of $get_{message}$(); after the function returns, the frame disappears, and the pointer becomes invalid (dangling) — puts(string) reads memory that no longer exists. Compilers usually warn about this. Solutions: a static array, a buffer passed by the caller, or malloc'd memory that the caller will free.

S5. Why doesn't swap(int a, int b) change x and y in main, and how to fix it?

Answer. C passes copies of values: swap changes its own local copies, the variables in main are untouched. The fix — pointers: void swap(int *x, int* y) { int temp = *x;* x = *y;* y = temp; } with the call swap(&a, &b); after it a = 7, b = 5.

S6. What does pa + i give and why is it not simple addition of numbers?

Answer. pa + i points to arr[i]: the address increases by i multiplied by the size of the data type. A pointer is not an int: pointer arithmetic is scaled by the element size (for int, the step is 4 bytes on a 64-bit platform).

S7. If arr[0] has address 100 (an int array), what is the address of arr[3]? And with char *pc = (char*)pa — what i gives (int *)(pc + i) == pa + 3?

Answer. arr[3] has address 112: 100 + 3 × 4 bytes (size of int). For a char pointer, the step is 1 byte, so i = 12: 3 ints = 12 bytes.

S8. What string functions are in string.h?

Answer. Copying: strcpy(strto, strfrom), strncpy(strto, strfrom, n). Comparison: strcmp(str1, str2) — 0 for equality, >0 if str1 > str2, <0 if str1 < str2; strncmp — first n characters. Length: strlen(str). Concatenation: strcat, strncat. Character search: strchr — first occurrence, strrchr — last (NULL if not found).

S9. Name the complexities of the four algorithms from lecture 7.

Answer. Insertion sort — $O(n^2)$. Quicksort — O(n log n) on average, $O(n^2)$ in the worst case (implemented as qsort() in stdlib.h). Linear search — visits on average half of the elements. Binary search — O(log n) on average and in the worst case (requires sorted data and random access; implemented as bsearch()).

S10. What are the three ingredients of recursion?

Answer. (1) Base case — the answer without another call (n == 0). (2) Smaller problem — the same function on a simpler input ($count_{down}$(n - 1)). (3) Progress — every valid recursive path approaches the base case (3, 2, 1, 0).

S11. What does $count_{down}$(3) and $count_{up}$(3) print and why is the output different?

Answer. $count_{down}$(3) prints 3 2 1, $count_{up}$(3) prints 1 2 3. In $count_{down}$, the print comes before the recursive call — values appear on the way down. In $count_{up}$, the print comes after the call — it waits for the smaller call to return and executes on the way back. Both functions visit n = 0 and stop.

S12. Why does $count_{down}$ with a recursive call $count_{down}$(n) loop infinitely and how to fix it?

Answer. The input never reaches 0: each call gets the same n, printing 3, 3, 3, 3… — the existence of a base case in the code is insufficient, every path must make progress. Fix: $count_{down}$(n - 1). Unbounded recursion can exhaust the stack space.

S13. What does fibonacci(4) return and why does fibonacci(2) appear twice?

Answer. fibonacci(4) returns 3. The call tree contains nine calls: 4, 3, 2, 1, 0, 1, 2, 1, 0 — both fibonacci(2) subtrees compute the same answer independently: the second branch doesn't remember the first. Only one call executes at any moment; the simple version repeats work.

S14. Tower of Hanoi: name the three recursive steps and the total number of moves for three disks.

Answer. (1) Move 2 disks from A to B using C — 3 moves; (2) move disk 3 from A to C — 1 move; (3) move 2 disks from B to C using A — 3 moves. Total 3 + 1 + 3 = 7 moves. Base case: no disks — no moves (n == 0). Rules: one disk at a time, only the top disk, never a larger on a smaller.

5.3. Week 7 Tests

T1. What does the & operator do when applied to a variable?

a) returns the address of the variable
b) dereferences a pointer
c) multiplies the variable by itself
d) declares a pointer

Answer: a) returns the address of the variable

*Explanation: "How to find an address of a variable? The & operator": int *pn = &n; — address of integer n.*

T2. What type does the expression &n have if int n;?

a) int
b) int *
c) double *
d) void

Answer: b) int *

Explanation: "Address of a variable of type t has type t " — address of int has type int .

T3. What does *ppi* = ppi + *pn; do?

a) changes the value of the variable pi through a pointer
b) changes the address of ppi
c) causes a compilation error
d) does nothing

Answer: a) changes the value of the variable pi through a pointer

Explanation: left side is writing to an address (pi is now 7.14159), right side is reading values through dereferencing; a dereferenced pointer behaves like a regular variable.

T4. Which of the following does NOT have an address?

a) a variable in memory
b) register variable
c) array element
d) pointer

Answer: b) register variable

Explanation: "What doesn't have an address? register variables; constants/literals/preprocessor defines; expressions (unless result is a variable)".

T5. What is sizeof(arr) for int arr[8]; on a 64-bit platform?

a) 8
b) 32
c) 4
d) 64

Answer: b) 32

Explanation: for primitive arrays sizeof gives the size of the array in bytes: 8 elements × 4 bytes = 32; long arr[5] gives 40.

T6. Where do declared variables go, and where does dynamic memory go?

a) both in the stack
b) both in the heap
c) declared — in the stack, dynamic — in the heap
d) declared — in the heap, dynamic — in the stack

Answer: c) declared — in the stack, dynamic — in the heap

Explanation: "Usable parts of virtual memory: stack and heap. stack: where declared variables go; heap: where dynamic memory goes".

T7. strcmp(str1, str2) returns 0 when…

a) str1 is greater than str2
b) the strings are equal
c) str1 is less than str2
d) the strings are of different lengths

Answer: b) the strings are equal

Explanation: strcmp returns 0 if equal; positive if str1 > str2; negative if str1 < str2.

T8. What does strchr(str, c) do?

a) searches for a substring in a string
b) returns a pointer to the first occurrence of character c or NULL
c) returns the last occurrence of c
d) calculates the length of the string

Answer: b) returns a pointer to the first occurrence of character c or NULL

Explanation: strchr is first occurrence, strrchr is last; both return NULL if the character is not found.

T9. The base case of recursion is…

a) an answer that is obtained without a new recursive call
b) the first function call
c) a loop inside the function
d) the last return from main

Answer: a) an answer that is obtained without a new recursive call

Explanation: "Base case: an answer without another call" — in $count_{down}$ this is n == 0: return;. Other ingredients: smaller problem and progress.

T10. What does $count_{down}(3)$ print?

a) 3 2 1
b) 1 2 3
c) 3 3 3
d) 0 1 2 3

Answer: a) 3 2 1

Explanation: the print is before the recursive call — values appear on the way down; $count_{up}(3)$ with print after the call gives 1 2 3.

5.4. Assignments — Week 7 Glossary

Term (atama)	Meaning
Pointer	address of a variable in memory; access/modify a variable from anywhere; basis of data structures
& operator (address-of)	returns the address of a variable; address of a variable of type t has type t *
Dereferencing (*) (dereferencing/indirection)	reading/writing a value at an address: *ppi* = ppi + *pn;
Null pointer (NULL)	a pointer 0 that doesn't point to anything
Virtual memory	OS abstraction: address space available to code; mapped to physical memory
Stack	part of virtual memory where declared variables go (and call frames)
Heap	where dynamic memory goes
Dangling pointer	a pointer to a variable that has gone out of scope (returning address of a local array)
Address arithmetic	pa + i points to arr[i]; address increases by i × size of type; arr[0] at address 100 → arr[3] at 112
Null terminator ('\0') and strlen	a zero byte that terminates a C-string; strlen(str) returns the length without the terminator — for copying, you need a buffer of +1 byte
Base case	answer without a new recursive call: n == 0 → return 1 in factorial; two base cases for fibonacci
Call stack / frame (stack frame, activation record)	record of an incomplete call: own parameters, local variables, continuation point
O-notation (Big-O)	order of growth of an algorithm: insertion $O(n^2)$; quicksort O(n log n) on average, $O(n^2)$ in the worst case; binary search O(log n)
Tower of Hanoi	classic recursive problem: 3 disks from A to C via B in 3 + 1 + 3 = 7 moves; base case — no disks, no moves

6. Bug Hunts — all errors and their fixes

Three course worksheets — "bug hunts": **bug_{huntc}.txt** (conditionals and functions), **$bug_{huntloops}$.txt** (loops), and **$bug_{huntpointersstrings}$.txt** (pointers and strings, with instructor key $bug_{huntpointersstringsanswerkey}$.txt). Game rules from worksheets: work in pairs; in each program find an error, explain why it's wrong, and fix it; compile and test with suggested inputs. Key tip: compile with gcc -g -Wall -fsanitize=address and run fixes — sanitizer messages are great discussion material. Below each task is fully analyzed: verbatim listing, explanation, and corrected code.

6.1. Bug Hunt C

"BUG HUNT: CONDITIONALS AND FUNCTIONS (C)" worksheet — six tasks. Quick round (10 minutes): tasks 1, 2, 5; tasks for discussion: 3 and 4 (errors only visible on specific inputs); C-specific lessons: 2, 5, and 6 (compiler errors and uninitialized variables). Instructor key included in file and provided below.

B1. Grade checker — condition order

Expected: 95 → 'A', 85 → 'B', 75 → 'C', 65 → 'D', 40 → 'F'. Flawed listing (verbatim):

```
#include

char grade(int score) {
     if (score >= 90) {
        return 'A';
     else if (score >= 80) {
        return 'D';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'B';
    } else
    } else {
        return 'F';
    }
}

int main(void) {
char student1 = grade(95);
char student2 = grade(78
    printf("%c\n", grade(95));
    puts("hello");
    return 0;
}
```

B1. What's wrong with the grade program and how to fix it?

Answer. Per key: conditions are checked in wrong order — highest boundary should come first (>= 90, then >= 80, then >= 70, then >= 60); due to wrong order score = 95 first enters (score >= 60) branch and returns 'D', other branches unreachable. In printed listing order is mixed with letters (>= 80 branch returns 'D', >= 60 returns 'B') and syntax is broken: missing closing parenthesis after return 'A'; duplicated else; grade(78 call not closed — program won't even compile. Fix: check boundaries in descending order, return each branch its letter, and restore parentheses:

```
#include

char grade(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'D';
    } else {
        return 'F';
    }
}
```

```
int main(void) {
    char student1 = grade(95);
    char student2 = grade(78);
    printf("%c\n", grade(95));
    puts("hello");
    return 0;
}
```

[Source: bug$_{huntc}$.txt, PROBLEM 1 + key]

B2. Even or odd — comparison and lost return

Expected: is$_{even}$(4) → 1, is$_{even}$(7) → 0. Flawed listing (verbatim):

```
#include

int is_even(int n) {
    int result;
    if (n % 2 == 0) {
        result = 0;
    } else {
        result = 1;
    }
    return result;
}


int main(void) {
result = 5;
    printf("%d\n", is_even(4));
    return 0;
}
```

B2. What's wrong with the is$_{even}$ program and how to fix it?

Answer. Key names two errors: (a) "n % 2 = 0" — assignment instead of comparison, need == (GCC reports error: cannot assign to expression); (b) function doesn't return result — missing return result; behavior undefined. In printed listing comparison already fixed to ==, but branches swapped: even number gets 0, odd gets 1, and main accesses result outside function — no result variable in main. Correct: even → 1, odd → 0, return result, remove result = 5; from main:

```
#include

int is_even(int n) {
    int result;
    if (n % 2 == 0) {
        result = 1;
    } else {
        result = 0;
    }
    return result;
}

int main(void) {
    printf("%d\n", is_even(4));
    return 0;
}
```

[Source: bug$_{huntc}$.txt, PROBLEM 2 + key]

B3. Leap year — unreachable % 400 branch

Expected: 1900 → 0, 2000 → 1, 2024 → 1, 2023 → 0. Listing (verbatim):

```
#include

int is_leap(int year) {
    if (year % 4 == 0) {
        return 1;
    } else if (year % 100 == 0) {
        return 0;
    } else if (year % 400 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int main(void) {
    printf("%d\n", is_leap(1900));
    printf("%d\n", is_leap(2000));
    return 0;
}
```

B3. Why does is_{leap}(1900) return 1, though 1900 is a common year?

Answer. Any year divisible by 4 immediately returns 1, so 1900 is incorrectly considered leap, and % 400 branch can never execute — it comes after % 4. Key prescribes: check % 400 first, then % 100, then % 4 — or use compact expression (same condition as in week 4 lab): return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;.

```
#include

int is_leap(int year) {
    if (year % 400 == 0) {
        return 1;
    } else if (year % 100 == 0) {
        return 0;
    } else if (year % 4 == 0) {
        return 1;
    } else {
        return 0;
    }
}

/* or in one line, according to week 4 lab: */
int is_leap(int year) {
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}
```

[Source: bug_{huntc}.txt, PROBLEM 3 + key]

B4. Biggest of three — equality of largest values

Expected: biggest(5, 5, 3) → 5. Listing (verbatim):

```
#include

int biggest(int a, int b, int c) {
    if (a > b && a > c) {
        return a;
    } else if (b > a && b > c) {
        return b;
    } else {
```

```
        return c;
    }
}

int main(void) {
    printf("%d\n", biggest(5, 5, 3));
    printf("hello");
    puts("hello")
    return 0;
}
```

B4. Why does biggest(5, 5, 3) return 3?

Answer. When the maximum values are equal (5 and 5), no strict ">" comparison is true: both initial conditions are false, execution falls through to else and returns c = 3. Key: use >= in comparisons. Additionally, there's a missing semicolon after puts("hello") in main — a syntax error that also needs to be fixed:

```
#include

int biggest(int a, int b, int c) {
    if (a >= b && a >= c) {
        return a;
    } else if (b >= a && b >= c) {
        return b;
    } else {
        return c;
    }
}

int main(void) {
    printf("%d\n", biggest(5, 5, 3));
    printf("hello");
    puts("hello");
    return 0;
}
```

[Source: bug$_{huntc}$.txt, PROBLEM 4 + key]

B5. Double and add — void-function in expression

Expected: total = 16 (10 + 6). Listing (verbatim):

```
#include

void doubled(int x) {
    printf("%d\n", x * 2);
}

int main(void) {
    int total = doubled(5) + doubled(3);
    printf("%d\n", total);
    return 0;
}
```

B5. Why doesn't int total = doubled(5) + doubled(3); compile?

Answer. doubled is declared as void — it returns nothing and cannot be used in an expression; it prints instead of returning. The compiler reports: "void value not ignored as it ought to be". Key: change the return type to int and use return x * 2;, move the printing to main. Then total = 10 + 6 = 16, as expected:

```
#include
```

```
int doubled(int x) {
    return x * 2;
}

int main(void) {
    int total = doubled(5) + doubled(3);
    printf("%d\n", total);
    return 0;
}
```

[Source: bug$_{huntc}$.txt, PROBLEM 5 + key]

B6. Member discount — uninitialized variable

Expected: apply$_{discount}$(100, 0) → 100.0, apply$_{discount}$(100, 1) → 80.0. Listing (verbatim):

```
#include

double apply_discount(double price, int is_member) {
    double discount;
    if (is_member) {
        discount = 0.2;
    }
    return price * (1 - discount);
}

int main(void) {
    printf("%.2f\n", apply_discount(100, 0));
    return 0;
}
```

B6. What value does discount have when is$_{member}$ = 0 and why is this dangerous?

Answer. When is$_{member}$ = 0, the if branch is not executed, and discount remains uninitialized — the result will be garbage (undefined behavior). This is a C-specific pitfall: C has no default initialization. Key: initialize the variable (double discount = 0.0;) or add an else branch:

```
#include

double apply_discount(double price, int is_member) {
    double discount = 0.0;
    if (is_member) {
        discount = 0.2;
    }
    return price * (1 - discount);
}

int main(void) {
    printf("%.2f\n", apply_discount(100, 0));
    return 0;
}
```

[Source: bug$_{huntc}$.txt, PROBLEM 6 + key]

6.2. Bug Hunt Loops

The sheet is titled "C++ BUG HUNT: LOOPS" and assumes #include <iostream>, <vector> and using namespace std; fragments use both printf and cout — the listing is preserved verbatim. Sheet rules: each program contains exactly ONE error related to loops; read the expected behavior; find and underline the line; explain in one sentence; write the fix. There is no official key in the file — the analyses below are compiled

based on the expected behavior of each fragment.

L1. Sum of grades — verification task

Expected: prints "Sum: 405". Listing (verbatim):

```
    int scores[5] = {90, 85, 70, 100, 60};
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += scores[i];
    }
    printf("Sum: %d", sum);
```

L1. Find the error in this loop.

Answer. This is a verification task: as written, the loop is correct — i passes through indices 0–4, each element is added once, and the sum 90 + 85 + 70 + 100 + 60 = 405, so the program prints exactly "Sum: 405", as expected. Lesson: before "fixing" working code, calculate the result manually — not every fragment with a loop contains a bug (the only formal issue is the missing newline \n in printf, which doesn't affect the result). No fix needed:

```
int scores[5] = {90, 85, 70, 100, 60};
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += scores[i];
    }
    printf("Sum: %d\n", sum); /* added \n — optional */
```

[Source: bug$_{huntloops}$.txt, BUG 1]

L2. Semicolon after for

Expected: prints the sum 1..10, i.e., 55. Listing (verbatim):

```
    int sum = 0;
    int i;
    for (i = 1; i <= 10; i++); {
        sum += i;
  }
    printf("%d", sum);
```

L2. Why is 11 printed instead of 55?

Answer. The semicolon immediately after the for header makes the loop body an empty statement: the loop "runs" ten times empty, bringing i to 11, and only then is the block { sum += i; } executed once with i = 11 — sum = 11. Fix: remove ';' after for (i = 1; i <= 10; i++):

```
    int sum = 0;
    int i;
    for (i = 1; i <= 10; i++) {
        sum += i;
    }
    printf("%d", sum);
```

[Source: bug$_{huntloops}$.txt, BUG 2]

L3. Missing opening brace

Expected: prints "Count: 0" … "Count: 4" and stops. Listing (verbatim):

```
    for (int count = 0; count < 5; ++count)
        printf("Count: %d", count);
    }
```

L3. What prevents this program from printing Count: 0 … 4?

Answer. There is no opening '{' after the for header, so the loop body is only the printf — by itself this would give correct output, but the lone '}' at the end is a syntax error: the program won't compile. Fix: add '{' after the header (or remove the extra '}'):

```
for (int count = 0; count < 5; ++count) {
    printf("Count: %d", count);
}
```

[Source: bug$_{huntloops}$.txt, BUG 3]

L4. printf without argument

Expected: prints "1 2 3 4 Found 5!" and stops. Listing (verbatim):

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        printf("Found 5!");
        break;
    }
    printf("%i ");
}
```

L4. Why don't the expected numbers 1 2 3 4 appear?

Answer. In printf("%i "); there is a format specifier %i, but no argument is passed — behavior is undefined (usually garbage is printed instead of numbers). Fix: pass i — printf("%i ", i);. Then the loop will print 1 2 3 4, when i == 5 — "Found 5!" and break will terminate the loop:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        printf("Found 5!");
        break;
    }
    printf("%i ", i);
}
```

[Source: bug$_{huntloops}$.txt, BUG 4]

L5. Unsigned counter can't be negative

Expected: prints "5 4 3 2 1 0 Liftoff!" and the program terminates. Listing (verbatim):

```
for (unsigned int i = 5; i >= 0; i--) {
    printf("%d ", i);
}
printf("Liftoff!");
```

L5. Why is "Liftoff!" never printed?

Answer. i is declared as unsigned int — it cannot be less than zero, so the condition i >= 0 is always true: after 0, the i-- operator "wraps" the counter to a huge number (principle of unsigned arithmetic from week 4: unsigned wraparound), and the loop becomes infinite. Fix: use a signed int counter, for which the condition i >= 0 will become false after 0:

```
for (int i = 5; i >= 0; i--) {
    printf("%d ", i);
}
printf("Liftoff!");
```

[Source: bug$_{huntloops}$.txt, BUG 5]

L6. erase during iteration (C++)

Expected: remove all even numbers, leaving {1, 3, 5}. Listing (verbatim):

```
vector nums = {1, 2, 3, 4, 5, 6};
for (auto it = nums.begin(); it != nums.end(); ++it) {
    if (*it % 2 == 0) {
        nums.erase(it);
    }
}
for (int n : nums) {
    cout << n << " ";
}
cout << endl;
```

L6. What happens to the iterator after nums.erase(it)?

Answer. nums.erase(it) makes the iterator it invalid (iterator invalidation), and the loop immediately does ++it on an invalid iterator — undefined behavior. Fix: assign the iterator returned by erase (it points to the next element), and increment it only when no deletion occurred:

```
vector nums = {1, 2, 3, 4, 5, 6};
for (auto it = nums.begin(); it != nums.end(); ) {
    if (*it % 2 == 0) {
        it = nums.erase(it);
    } else {
        ++it;
    }
}
for (int n : nums) {
    cout << n << " ";
}
cout << endl;
```

[Source: $bug_{huntloops}$.txt, BUG 6]

L7. Comparing double for equality

Expected: prints 0, 0.1, 0.2, … up to 0.9 and stops. Listing (verbatim):

```
for (double x = 0.0; x != 1.0; x += 0.1) {
    printf("%f ", x);
}
```

L7. Why doesn't the loop stop at 0.9?

Answer. 0.1 cannot be represented exactly in binary floating-point arithmetic; after ten additions, the accumulated error "misses" the exact 1.0, the condition x != 1.0 remains true, and the loop goes beyond 0.9 (same rule as in week 4: == comparison on float/double is unreliable due to finite precision). Fix: use a relational condition x < 1.0:

```
for (double x = 0.0; x < 1.0; x += 0.1) {
    printf("%f ", x);
}
```

[Source: $bug_{huntloops}$.txt, BUG 7]

L8. Variable shadowing of total

Expected: prints "Total: 15". Listing (verbatim):

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    int total = 0;
    total += i;
}
```

```
printf("Total: %d", total);
```

L8. Why does it print "Total: 0" instead of 15?

Answer. Inside the loop, a new variable total is declared, which shadows the outer one (shadowing — same mechanism as with local nmax = 25 from lecture 5): each iteration creates a fresh inner total, adds i to it, and discards it. The outer total is never modified. Fix: remove the inner declaration int total = 0; — then total += i; will update the outer variable: 1 + 2 + 3 + 4 + 5 = 15:

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    total += i;
}
printf("Total: %d", total);
```

[Source: bug$_{huntloops}$.txt, BUG 8]

L9. Incrementing the wrong variable

Expected: 3×3 multiplication table: rows 1 2 3 / 2 4 6 / 3 6 9. Listing (verbatim):

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; i++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

L9. Why does the inner loop never end?

Answer. The inner loop increments i instead of j: j remains 1 forever, the condition j <= 3 is always true, and i grows infinitely — the inner loop loops infinitely (and messes up the outer counter). Fix: j++ in the inner header:

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

[Source: bug$_{huntloops}$.txt, BUG 9]

L10. Finding 7 in a grid — truncated listing [Hard]

Expected: find the FIRST occurrence of 7 in the grid and print "First 7 found in row 0". The listing in the assignment file is truncated (verbatim, entire available fragment):

```
int grid[3][3] = {
    {1, 7, 3},
```

L10. The listing is truncated — what lesson is embedded in this problem?

Answer. The complete code for the problem is missing from the course file (the text breaks off right after the start of the grid declaration), so a verbatim fix is impossible. From the visible fragment: 7 is in row 0, column 1 — expected output "First 7 found in row 0". The rule this problem tests: break only exits the innermost loop (lecture 5: "break exits innermost loop or switch") — when searching a grid, break in the inner column loop won't stop the outer row loop. To terminate the search after the first find, use a "found" flag or goto-exit from nested blocks (lecture 6: "goto provides a convenient way to exit from nested blocks"). Treat this problem as a discussion of rules, not as a compilable fragment.

[Source: bug$_{huntloops}$.txt, BUG 10 — fragment truncated in the original file]

6.3. Bug Hunt Pointers & Strings

The "C BUG HUNT: POINTERS AND STRINGS" sheet — twelve problems. All errors in the key have been verified by compilation with gcc and running under AddressSanitizer/UBSan; many are undefined behavior, so exact symptoms may differ across compilers, flags, and machines. The instructor key (bug$_{huntpointersstringsanswerkey}$.txt) is the authoritative source for fixes; below each problem: listing verbatim, explanation from the key, and corrected code.

P1. Missing null terminator

```
char s[5] = {'h', 'e', 'l', 'l', 'o'};
printf("%s\n", s);
```

P1. Expected "hello" — what will this code print and why?

Answer. In s there are exactly 5 cells for 5 letters — no space remains for '\0'. printf("%s") continues reading memory beyond the array until it encounters a zero byte: the output is unpredictable. Fix by key: char s[6] = {'h','e','l','l','o','\0'}; or char s[] = "hello"; (the compiler will add the terminator and size itself):

```
char s[6] = {'h', 'e', 'l', 'l', 'o', '\0'};
printf("%s\n", s);

/* or: */
char s2[] = "hello";
printf("%s\n", s2);
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 1 + key]

P2. sizeof on a pointer

```
char *s = "hello";
printf("%zu\n", sizeof(s));
```

P2. Expected string length is 5 — what will sizeof(s) return?

Answer. s is a char *, so sizeof(s) is the size of the pointer itself (8 on a 64-bit machine), not the string length: 8 will be printed. strlen(s) returns the string length. Key advice: sizeof on a real array (char a[] = "hello") gives 6, including '\0'.

```
char *s = "hello";
printf("%zu\n", strlen(s));
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 2 + key]

P3. String comparison via ==

```
char a[] = "cat";
char b[] = "cat";
if (a == b)
    puts("Same");
else
    puts("Different");
```

P3. Why is "Different" printed, although the text is identical?

Answer. The == operator compares the addresses of the two arrays — these are two different objects in memory, so the comparison is false. The string contents are compared by strcmp: if (strcmp(a, b) == 0) — then "Same" will be printed.

```
char a[] = "cat";
char b[] = "cat";
if (strcmp(a, b) == 0)
```

```
        puts("Same");
    else
        puts("Different");
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 3 + key]

P4. Buffer too small for strcpy

```
char dest[5];
strcpy(dest, "hello");
printf("%s\n", dest);
```

P4. How many bytes does dest need to copy "hello" and what happens here?

Answer. "hello" requires 6 bytes: 5 letters plus '\0', while dest holds 5 — strcpy writes one byte beyond the array end (buffer overflow). Fix: char dest[6]; or larger.

```
char dest[6];
strcpy(dest, "hello");
printf("%s\n", dest);
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 4 + key]

P5. Modifying a string literal

```
char *s = "hello";
s[0] = 'J';
printf("%s\n", s);
```

P5. Why does s[0] = 'J'; cause the program to crash?

Answer. "hello" is a read-only literal in memory; s merely points to it, and s[0] = 'J' attempts to modify read-only memory — typically a segmentation fault (segfault). Fix by key: char s[] = "hello"; — this creates a modifiable copy in an array, and the program prints "Jello".

```
char s[] = "hello";
s[0] = 'J';
printf("%s\n", s);
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 5 + key]

P6. Returning a pointer to a local variable

```
char *greeting(void) {
    char buf[20];
    strcpy(buf, "Hi there");
    return buf;
}

int main(void) {
    char *g = greeting();
    printf("%s\n", g);
    return 0;
}
```

P6. Why does the pointer g "dangle" after calling greeting()?

Answer. buf lives on the greeting() stack frame, which disappears immediately after the function returns — g becomes a dangling pointer; reading *g is undefined behavior. Compilers usually warn. Fixes by key: a static array, a buffer passed by the caller, or malloc'd memory that the caller will free. Static variant:

```
char *greeting(void) {
    static char buf[20];
    strcpy(buf, "Hi there");
```

```
    return buf;
}

int main(void) {
    char *g = greeting();
    printf("%s\n", g);
    return 0;
}
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 6 + key]

P7. Uninitialized (wild) pointer

```
int *p; // -> Memory address
*p = 10;
printf("%d\n", *p);
```

P7. Where does *p = 10 write and why is this dangerous, even if 10 is printed?

Answer. p was never assigned — it contains a garbage address, and *p = 10 writes to an unknown address: the program may crash, or it may "work" — a dangerous case (in the key test, 10 was printed by pure chance). Fix by key: point p to real memory — int v; int* p = &v; or p = malloc(sizeof(int)):

```
int v;
int *p = &v;
*p = 10;
printf("%d\n", *p);
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 7 + key]

P8. Pass by value

```
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main(void) {
    int x = 1, y = 2;
    swap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

P8. Why after swap(x, y) are "1 2" printed, not "2 1"?

Answer. C passes copies: swap swaps its own local copies of x and y, while the main variables remain untouched. Fix by key — the same function with pointers from lecture 7: void swap(int *a, int* b) and call swap(&x, &y):

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int main(void) {
    int x = 1, y = 2;
    swap(&x;, &y;);
    printf("%d %d\n", x, y);
    return 0;
}
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 8 + key]

P9. malloc without +1 for '\0'

```
char *s = "hello";
char *copy = malloc(strlen(s));
strcpy(copy, s);
printf("%s\n", copy);
free(copy);
```

P9. How many bytes does the heap block overflow by and why?

Answer. strlen doesn't count the terminator: for "hello", malloc(strlen(s)) allocates 5 bytes, while strcpy writes 6 (5 letters + '\0') — the heap block overflows by one byte. Fix by key: malloc(strlen(s) + 1).

```
char *s = "hello";
char *copy = malloc(strlen(s) + 1);
strcpy(copy, s);
printf("%s\n", copy);
free(copy);
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 9 + key]

P10. Unsigned counter in reverse loop

```
char s[] = "abc";
for (size_t i = strlen(s) - 1; i >= 0; i--) {
    putchar(s[i]);
}
putchar('\n');
```

P10. Why doesn't the "reverse" loop stop after 'a'?

Answer. size$_t$ is an unsigned type: the condition i >= 0 is always true. After i = 0, the decrement wraps i around to a huge number, and s[i] reads far beyond the array bounds. Two fixes from the key: a signed counter or a loop with i > 0 and access to s[i - 1]:

```
char s[] = "abc";
for (int i = (int)strlen(s) - 1; i >= 0; i--) {
    putchar(s[i]);
}
putchar('\n');

/* or: */
char t[] = "abc";
for (size_t i = strlen(t); i > 0; i--) {
    putchar(t[i - 1]);
}
putchar('\n');
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 10 + key]

P11. Missing & in scanf

```
int x;
scanf("%d", x);
printf("%d\n", x);
```

P11. What does scanf("%d", x) do without &?

Answer. scanf needs the address of a variable; passing x (uninitialized int) makes scanf treat its garbage value as a pointer and write to that address — undefined behavior; most compilers warn with -Wall. Fix: scanf("%d",

&x);.

```
int x;
scanf("%d", &x;);
printf("%d\n", x);
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 11 + key]

P12. Use after free

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p);
```

P12. Why printing *p after free(p) is an error?

Answer. After free(p), the memory may already be reused: reading *p may print 42, garbage, or crash (use after free — undefined behavior). By key: don't touch* p after free; print before freeing; after free assign p = NULL to make the error loud:

```
int *p = malloc(sizeof(int));
*p = 42;
printf("%d\n", *p);
free(p);
p = NULL;
```

[Source: bug$_{huntpointersstrings}$.txt, BUG 12 + key]

Bug map of the sheet (from key)

Error type	Problem numbers
Missing null terminator / +1 for '\0'	1, 4, 9
Pointer and array confusion	2, 3
Read-only memory / dangling memory	5, 6, 12
Uninitialized pointers	7, 11
Pass by value	8
Unsigned wraparound	10

Quick sort by bug type [Source: bug$_{huntpointersstringsanswerkey}$.txt]

7. Cheat sheet: key rules and tables

Final chapter — everything you need before the test and exam: operator precedence, complete printf/scanf format table, key rules of C weeks 4–7 on one page, ten answers in one line, and what's next in the course.

7.1. Operator precedence

Precedence	Operators	Comment
1 (highest)	++ – –, (cast), sizeof	unary operators, type casting, size
2	*, /, %	multiplicative — higher than + and –
3	+, –	additive
4	==, !=	equality comparison — higher than && and ||

Precedence	Operators	Comment
5	&&, ||	logical
6 (lowest)	=, +=, −=, *=, /=, %=	assignment — very low priority

Precedence and evaluation order [Source: lecture$_4$, slide 25]

Slide rule: use parentheses ( ) generously to avoid ambiguities and side effects of precedence. Examples of equivalence:

```
y = x*3+2                 /* same as y = (x*3)+2 */
x != 0 && y == 0          /* same as (x != 0) && (y == 0) */
d = c >= '0' && c <= '9'  /* same as d = (c >= '0') && (c <= '9') */
```

7.2. printf/scanf formats

Complete format specification: **%[flags][width][.precision][length]<type>**. Type specifiers (lectures 6 and 7):

Specifier	Type	Example / output
d, i	int	printf("%d",10); → 10
u	unsigned int	printf("%u",10); → 10
o	unsigned int (octal)	from lecture 7: u, o, x, X — unsigned int
x, X	unsigned int (hexadecimal)	printf("%x",10); → 0xa
c	char	printf("%c",'A'); → A
s	string	printf("%s","hello"); → hello
f	float/double	printf("%f",2.3); → 2.3
e, E	real (exponent)	1e3, 1.2E3, 1E−3
F, g, G	double	from lecture 7: e, E, f, F, g, G — double
%	literal %	printf("%d %%",10); → 10%

printf type specifiers [Source: Lecture$_6$, slide 12; Lecture$_{71}$, slide 3]

Modifier	Meaning
h	interpret as short; with i, d, o, u, x
l	interpret as long; with i, d, o, u, x
L	interpret as double; with e, f, g

Length modifiers [Source: Lecture$_6$, slide 16]

printf call	Output
printf("%d",10)	«10»
printf("%4d",10)	«bb10» (b — space; width)
printf("%7s","hello")	«bbhello»
printf("%d,%+d,%+d",10,-10)	«10,+10,-10» (flag +)
printf("%04d",10)	«0010» (flag 0)
printf("%-7s","hello")	«hellobb» (flag −: left-aligned)

printf call	Output
printf("%.2f,%.0f",1.141,1.141)	«1.14,1» (precision)
printf("%.2e,%.0e",1.141,100.00)	«1.14e+00,1e+02»
printf("%.4s","hello")	«hell»
printf("%.1s","hello")	«h»

width, flags, precision [Source: Lecture$_6$, slides 13–15]

scanf — input analog of printf with the same specification; ignores whitespace characters; returns number of read items or EOF; **arguments are addresses of variables (except array names)**:

printf	scanf
printf("%d",x)	scanf("%d",&x)
printf("%f",f)	scanf("%f",&f)
printf("%s",str)	scanf("%s",str) — & not needed
printf("%s",str)	scanf("%20s",str) — & not needed
printf("%s %s",fname,lname)	scanf("%20s %20s",fname,lname)

printf ↔ scanf [Source: Lecture$_6$, slide 20]

Character functions stdio.h: putchar(c) — character to standard output (returns character or EOF); getchar() — next character of standard input or EOF; getc(fp)/putc(c, fp) — same for stream; getchar() is getc(stdin), putchar(c) is putc(c, stdout). Line by line: fgets(line, maxlen, fp) reads a line (including newline), fputs(line, fp) writes; to files: fopen(name, mode) with modes "r", "w", "a" (plus "b" for binary), fclose(fp); stdin/stdout/stderr are also FILE*.

7.3. Key rules of C

- **Operand types determine division**: 4/3 → 1 (int), 4.0/3.0 → 1.333333; the receiver type (double fraction = 4/3) doesn't affect it — division happens before assignment.
- **Variable names**: letters, digits, and _; must start with a letter; no keywords; case-sensitive (x and X are different variables).
- **Size guarantees**: sizeof(char) == 1; sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long); unsigned has roughly twice the range; float precision — 6, double — 15 significant digits.
- **Truth and falsehood**: any non-zero value is true, 0 is false; condition is a numeric expression or pointer; in C99, bool is from <stdbool.h>.
- **== is comparison, = is assignment**; comparison == on float/double is unreliable due to finite precision (0.1 cannot be represented exactly).
- **Short-circuit**: && and || stop evaluation when the result is determined; side effects of the right operand may not execute.
- **break** exits the innermost loop or switch; **without break in case, execution falls through** to the next case without new checking.
- **continue** skips the rest of the inner loop body, jumping to the condition (in for — to the update).
- **Semicolon after for(...)** makes the loop body empty — the block after it will execute once.
- **Arguments are passed by value**: for a function to change a caller's variable, pass a pointer (swap(&x, &y)); a returned structure is a separate copy that the caller decides where to store.

- **A C string is a char array with '\0'**: strlen doesn't count the terminator; copying requires +1 byte (malloc(strlen(s) + 1)); %s reads until the null byte.
- **String literals are read-only**: char *s = "hello"; s[0] = 'J'; — segfault; a mutable copy is char s[] = "hello";.
- **scanf needs addresses**: scanf("%d", &x); a char array name is already an address: scanf("%s", str).
- **Uninitialized variables are garbage**: a local double discount without a value — undefined behavior; C has no default values.
- **Recursion = base case + smaller task + progress**: without progress ($count_{down}(n)$ instead of $count_{down}(n - 1)$) — infinite recursion and stack overflow.

7.4. Ten answers in one line

- 1. What will double fraction = 4/3 print with %f? — 1.000000: integer division gives 1 before conversion.
- 2. How many significant digits in float and double? — 6 and 15 (FLT_{DIG}, DBL_{DIG}).
- 3. Leap year? — year % 400 == 0 || (year % 4 == 0 && year % 100 != 0): 2000 — yes, 1900 and 2100 — no.
- 4. What without break in switch? — fall-through to the next case without checking (melody 'A#' prints C, then D).
- 5. How many times will do-while execute? — at least once: condition is checked after the body.
- 6. What does continue do? — skips the rest of the loop body and jumps to the condition.
- 7. How to swap x and y in main? — void swap(int *a,* *int* b) and call swap(&x, &y): passing by value won't work.
- 8. What will strcmp("aA", "aA") return? — 0: strings are equal; >0 — str1 is greater, <0 — less.
- 9. What will $count_{down}(3)$ and $count_{up}(3)$ print? — 3 2 1 and 1 2 3: printing before the call — on the way down, after — on the way up.
- 10. fibonacci(4)? — 3; two base cases (F(0)=0, F(1)=1), fibonacci(2) is computed twice.

7.5. What's next

Based on lecture materials after week 7, the course continues with topics announced directly in the slides: "More searching next week…" (lecture 7, slide on binary search — more about searching), "More about sizeof() next time…" (slide on sizeof) and "Implicit cast to/from void * also possible (more next week…)" (slide on pointer casting), and the module concept from lecture 5 extends to external libraries ("Same concept carries over to external libraries (next week…)"). Before the midterm, review weeks 4–7 with this guide in the following order: types and division (chapter 2) → conditions, switch, and loops (chapter 3) → functions, structures, and I/O (chapter 4) → pointers, strings, and recursion (chapter 5), then run all bug hunts (chapter 6) and this cheat sheet: three exit checks from week 4 (why 4/3 loses the fraction; why 2100 is a common year; why the boundary comes first) and three exit checks from week 7 (where does factorial stop; why n – 1 moves toward the base case; what does factorial(3) expect) — ready questions for self-check.